

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 面向对象程序设计基础

实验时间 2025 年 9 月 12.19.26 日 第 34 节

年级 2024 专业班级 计科1班

姓名 唐明强 学 号 20242122002

评分 [单击此处成绩](#)

王庆平

教师签名

上机一面向对象基础

一、上机目的

- 1、熟悉类的封装特性，能够说出为什么要封装以及如何实现封装。
- 2、掌握构造方法的定义和重载，能够独立定义构造方法，重载构造方法。
- 3、熟悉 this关键字，能够使用this调用成员属性、成员方法、构造方法。
- 4、熟悉 static关键字的使用，能够说出静态(属性、方法、代码块)的特点。

二、上机任务

1. 编写程序，演示如何实现类的封装。定义一个Student 类，对age 这属性进行封装，然后测试。

程序：

```
package shiyan6.exa01;

/**
 * 什么是封装？
 * 封装就是将对象中的实现细节隐藏。不被外界所直接访问。
 * 封装的好处？
 * 可以提高代码的安性
 */

public class Student {

    private int age; // 封装--私有化，用private修饰

    String name;

    public int getAge() {

        return age;

    }

    public void setAge(int age) {

        this.age = age;

    }

}
```

```

%
-----%
package shiyan6.exa01;

public class test01 {

    public static void main(String[] args) {

        Student stul = new Student();

        stul.setAge(18);

        stul.name = "xiaoming";

        System.out.println("名字: " + stul.name + "年龄: " +
stul.getAge());

    }

}

```

运行结果:

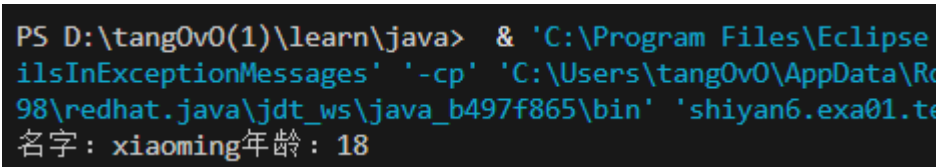


图 1 运行结果示意图

2. 编写程序，演示构造方法的定义，演示构造方法的重载， Person 类，进行测试。

程序:

```

package shiyan6.exa02;

/**
 * 构造方法的定义格式:
 * 权限修饰符方法名()方法体;
 * 1. 方法名和类名要保持一致
 * 2. 构造方法没有返回值类型，连void 都不能写
 * 3. 构造方法中不能写return语句
 * 构造方法的注意事项:
 * 1. 如果我们自己没有写任何的构造方法。系统会默认提供一个无参的构造方法。供我 们创
建对象使用。
 * 2. 如果自己写了一个有参的构造方法，那么系统就不会提供空参的构造方法了。
 * 构造方法重载:
 * 1. 只要每个构造方法的参数类型或参数个数不同即可。
 */

```

```

public class Person {
    String name;
    int age;

    public Person(String name) {
        this.name = name;
    }

    public Person(int age) {
        this.age = age;
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void speak() {
        System.out.println("我叫: " + name + " 今年" + age + " 岁了
!");
    }
}

```

```

package shiyan6.exa02;

public class test02 {
    public static void main(String[] args) {
        Person p1 = new Person(18);
        Person p2 = new Person("xiaoming");
        Person p3 = new Person("wqpxiaoming", 18);
        p3.speak();
    }
}

```

运行结果:

```
essages' '-cp' 'C:\Users\tang0v0\AppData\l
Storage\0db85fbb172ee92266fe9ed5ab435398\
7f865\bin' 'shiyang6.exe02.test02'
我叫: wqpxiaoming 今年18 岁了!
```

图2运行结果示意图

3. 编写程序，演示 this 关键字的使用，定义一个 Animal 类，然后测试。

程序:

```
package shiyang6.exe03;

/**
 * this 关键字:
 * 1. 可以区分局部变量和成员变量同名的问题
 * 2. 代表的是当前对象的引用。谁来调用我，我就代表谁。
 * 3. 在构造方法中可以调用本类中其他构造方法 this（参数），必须放在第一行
 * 4. 在成员方法中调用其他成员方法 this.方法名（），this 可以省掉不写。
 */

public class Animal {

    int age;

    String name;

    public Animal() {

        this(6, "tiger");

    }

    public Animal(int age, String name) {

        this.age = age;

        this.name = name;

    }

}
```

```

package shiyan6.exa03;

public class test {

    public static void main(String[] args) {

        Animal a1 = new Animal(8, "dog");

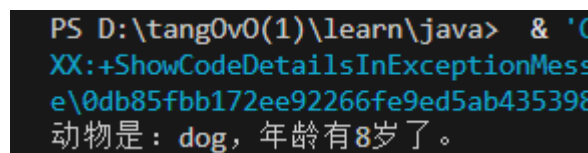
        System.out.println("动物是： " + a1.name + "，年龄有" + a1.age
+ "岁了。");

    }

}

```

运行结果：



```

PS D:\tang0v0(1)\learn\java> & 'C
XX:+ShowCodeDetailsInExceptionMess
e\0db85fbb172ee92266fe9ed5ab435398
动物是： dog，年龄有8岁了。

```

图3 运行结果示意图

4. 编写程序，演示 `static` 关键字的使用（修饰成员变量）。

程序：

```

package shiyan6.exa04;

/*
 * static 关键字：
 * 修饰变量，变量可以被所有的对象所共享。
 * 什么时候将变量定义成静态的呢？
 * 1. 当一个变量需要被类中的所有对象所共享的时候。
 * 2. 可以通过 类名.静态变量 访问静态变量。
 * 3. 只能修饰成员变量。
 */

public class shiyan {

    static int counter = 0;

    int id;

    public shiyan() {

```

```

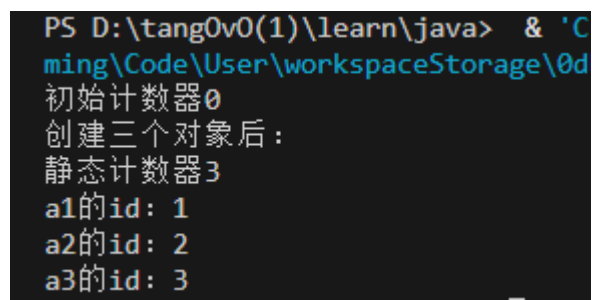
        counter++;

        id = counter;
    }

    public static void main(String[] args) {
        System.out.println("初始计数器" + shiyan.counter);
        shiyan a1 = new shiyan();
        shiyan a2 = new shiyan();
        shiyan a3 = new shiyan();
        System.out.println("创建三个对象后：");
        System.out.println("静态计数器" + shiyan.counter);
        System.out.println("a1的id: " + a1.id);
        System.out.println("a2的id: " + a2.id);
        System.out.println("a3的id: " + a3.id);
    }
}

```

运行结果：



```

PS D:\tang0v0(1)\learn\java> & 'C:\ming\Code\User\workspaceStorage\0d
初始计数器0
创建三个对象后:
静态计数器3
a1的id: 1
a2的id: 2
a3的id: 3

```

图4运行结果示意图

5. 编写程序，演示 static 关键字的使用（修饰方法）。

程序：

```
package shiyan6.exa04;
```

```
/*
```

- * 静态方法可以直接通过类名来调用
- * 静态属于类。随着类的加载而加载。
- * 非静态属于对象。随着对象的创建而加载。
- * 静态和非静态的访问特点：
 - * 1. 静态的只能访问静态的
 - * 2. 非静态的既可以访问静态的。也可以访问非静态的。

* 3. 静态方法多在工具类中。

*/

```
public class shiyan1 {  
    // 静态方法 - 可以直接通过类名调用  
    public static void printMessage() {  
        System.out.println("这是一个静态方法，无需创建对象即可调用");  
    }  
  
    // 实例方法 - 需要创建对象后才能调用  
    public void printInstanceMessage() {  
        System.out.println("这是一个实例方法，需要创建对象后才能调用");  
    }  
  
    // 另一个静态方法，演示静态方法之间的调用  
    public static void callAnotherStatic() {  
        System.out.println("这个静态方法调用了另一个静态方法:");  
        printMessage(); // 静态方法可以直接调用其他静态方法  
    }  
  
    public static void main(String[] args) {  
        // 直接通过类名调用静态方法，无需创建对象  
        shiyan1.printMessage();  
        System.out.println();  
  
        // 调用另一个静态方法  
        shiyan1.callAnotherStatic();  
        System.out.println();  
  
        // 实例方法需要创建对象后才能调用  
        shiyan1 obj = new shiyan1();  
        obj.printInstanceMessage();  
    }  
}
```


运行结果

```
e\0db85fbb172ee92266fe9ed5ab435398\redhat.jav
这是一个静态方法，无需创建对象即可调用

这个静态方法调用了另一个静态方法：
这是一个静态方法，无需创建对象即可调用

这是一个实例方法，需要创建对象后才能调用
PS D:\tangOv0(1)\learn\java>
```

图5 运行结果示意图

6、编写程序，学习子类是如何继承父类的。

程序：package shiyan6.exa05;

```
/**
 * 继承：
 * 1. 什么是继承？
 * 说白了就是让类与类之间产生了关系。子父类的关系
 * 2. 继承的关键字。
 * 子类 extends 父类
 * 3. java 中的继承的特点。
 * 只支持单继承。但是可以多层继承。
 * 4. 继承父类只能使用父类的公共成员。
 * 5. 继承的好处和弊端。
 * 好处：提高了代码的复用性。提高代码的维护性。
 * 弊端：类与类之间的耦合性太强。
 */
class Animal {
    String name;

    public Animal(String name) {
        this.name = name;
    }

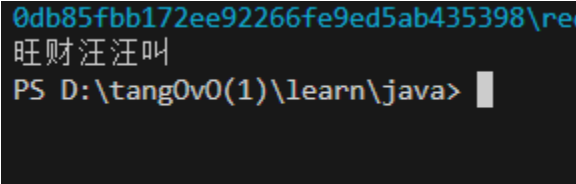
    public String speak() {
        return "动物叫";
    }
}
```

```
} // 父类：动物
```

```
class Dog extends Animal {  
    public Dog(String name) {  
        super(name);  
    }  
  
    public String speak() {  
        return name + "汪汪叫";  
    }  
}
```

```
public class exa {  
    public static void main(String[] args) {  
        Dog dog = new Dog("旺财");  
        System.out.println(dog.speak());  
    }  
}
```

运行结果：



```
0db85fbb172ee92266fe9ed5ab435398\re  
旺财汪汪叫  
PS D:\tang0v0(1)\learn\java>
```

图6运行结果示意图

7、编写程序，演示方法的重写。

程序: package shiyan6.exa06;

```
/**
```

- * 方法的重写-----为什么要进行方法的重写?
- * 方法的重写发生在子父类关系中。
- * 当父类提供的功能不满足子类具体的需求的时候，那么就需要对父类的方法进行重写。

```
*/
```

```
// 父类
```

```
class Animal {  
    public void makeSound() {  
        System.out.println("动物发出声音");  
    }  
}  
  
// 子类1: 重写父类方法  
class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("汪汪汪！");  
    }  
}  
  
// 子类2: 重写父类方法  
class Cat extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("喵喵喵！");  
    }  
}  
  
// 主类  
public class exa {  
    public static void main(String[] args) {  
        // 创建对象  
        Animal animal = new Animal();  
        Dog dog = new Dog();  
        Cat cat = new Cat();  
  
        // 调用方法，展示重写效果  
        System.out.println("动物声音：");  
        animal.makeSound();  
    }  
}
```

```

        System.out.println("\n狗的声音：");
        dog.makeSound();

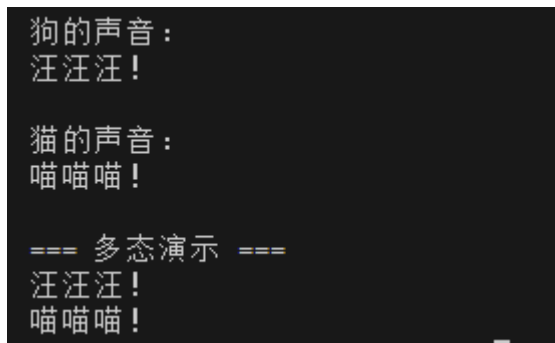
        System.out.println("\n猫的声音：");
        cat.makeSound();

        // 多态演示：父类引用指向子类对象
        System.out.println("\n=== 多态演示 ===");
        Animal myPet1 = new Dog();
        Animal myPet2 = new Cat();

        myPet1.makeSound(); // 输出：汪汪汪！
        myPet2.makeSound(); // 输出：喵喵喵！
    }
}

```

运行结果：



```

狗的声音：
汪汪汪！

猫的声音：
喵喵喵！

=== 多态演示 ===
汪汪汪！
喵喵喵！

```

图7 运行结果示意图

8、编写程序，重写 toString() 方法。

程序： package shiyan6.exa07;

```

/**
 * toString() 方法没有重写之前；
 * return getClass().getName() + '@' + Integer.toHexString(hashCode())
 * getClass() 获取当前运行对象的字节码对象。
 * getName() 获取包名和类名
 * @ 固定的连接符

```

```

* hashCode() 获取模拟出来的内存地址值
* Integer.toHexString(int num) 将十进制的整数转换成十六进制
*/

public class student {
    String name;
    int age;

    public student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String toString() {
        return "name=" + name + ",age=" + age;
    }
}

%-----%
package shiyan6.exa07;

public class exa {

    public static void main(String[]
args) {

        student stul = new
student("xixi", 18);

        System.out.println(stul);

        System.out.println(stul.toString());

    }

}

```

运行结果：

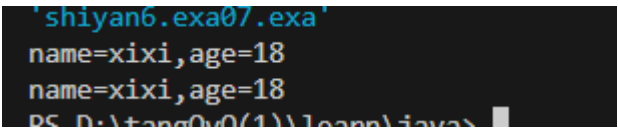


图8运行结果示意图

9、编写程序，演示 super 关键字的使用。

程序：package shiyang6.exe08;

```
/**
 * super 关键字
 * 可以访问父类的成员
 * 格式：
 * 1. 使用父类的成员变量：super.成员变量
 * 2. 使用父类的成员方法：super.成员方法([参数 1,参数 2...])
 * 3. 调用父类的构造方法：super([参数 1,参数 2...])
 */
public class Animal {
    String name;
    int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void sing() {
        System.out.println("动物会唱");
    }

    public void rap() {
        System.out.println("动物会rap");
    }
}
```

%-----%

```

package shiyan6.exa08;

public class Cat extends Animal {
    public Cat() {
        super("波斯猫", 1);
    }

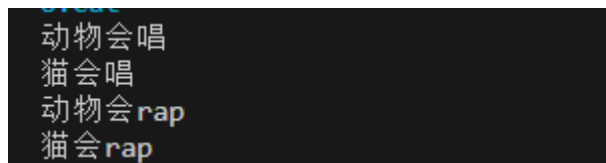
    public void sing() {
        super.sing();
        System.out.println("猫会唱");
    }

    public void rap() {
        super.rap();
        System.out.println("猫会rap");
    }

    public static void main(String[] args) {
        Cat cat = new Cat();
        cat.sing();
        cat.rap();
    }
}

```

运行结果：



```

create
动物会唱
猫会唱
动物会rap
猫会rap

```

图9运行结果示意图

10、编写程序，演示 final 关键字的使用。

程序： package shiyan6.exa09;

/**

* final 关键字作用：

* final 修饰类：类变成了一个最终的类，不能有任何的子类

```
* final 修饰方法：方法变成了一个最终的方法，不能重写
* final 修饰变量：变量就变成了一个常量，只能赋值一次
*/

// 1. final类：无法被继承
final class FinalClass {
    public void show() {
        System.out.println("FinalClass不能被继承");
    }
}

// 2. final方法：无法被重写
class Parent {
    public final void finalMethod() {
        System.out.println("父类的final方法，不能被子类重写");
    }
}

class Child extends Parent {
    // 尝试重写finalMethod()会导致编译错误
    // public void finalMethod() { }
}

public class FinalDemo {
    // 3. final变量：常量（必须初始化，不可修改）
    private final int finalVar = 100;
    private static final String CONSTANT = "全局常量";

    public void demonstrateFinal() {
        // final局部变量
        final double localFinal = 3.14;

        // 以下修改会编译错误：
        // finalVar = 200;
        // localFinal = 2.71;
    }
}
```



```

        System.out.println("实例常量: " + finalVar);
        System.out.println("静态常量: " + CONSTANT);
        System.out.println("局部常量: " + localFinal);
    }

    public static void main(String[] args) {
        FinalDemo demo = new FinalDemo();
        demo.demonstrateFinal();

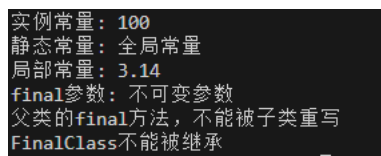
        // final参数示例
        demo.finalParameter("不可变参数");

        new Child().finalMethod(); // 调用继承的final方法
        new FinalClass().show();    // 使用final类
    }

    // 4. final参数: 方法内部不能修改参数值
    public void finalParameter(final String msg) {
        // msg = "修改"; // 编译错误
        System.out.println("final参数: " + msg);
    }
}

```

运行结果:



```

实例常量: 100
静态常量: 全局常量
局部常量: 3.14
final参数: 不可变参数
父类的final方法, 不能被子类重写
FinalClass不能被继承

```

图10运行结果示意图

11、编写程序，学习抽象类的使用

程序: package shiyan6.exa10;

/**

* 抽象类:

* 1. 什么是抽象类

* 当我们进行父类抽取的时候，有些方法具体每个子类的实现方式都不太一样。

* 那么这个时候，就应该把这个方法定义成抽象方法。
有抽象方法的类一定是抽象。

* 2. 抽象类和抽象方法如何定义

* `abstract`

* 3. 抽象类的成员特点

* 成员变量：既可以变量，也可以有常量。

* 成员方法： 即可有抽象方法，也可以有非抽象方法。

* 构造方法：可以有构造方法。

* 4. 抽象类的注意事项

* 抽象类不能实例化对象。

*/

```
public abstract class Animal {  
  
    String name;  
  
    String color;  
  
    int age;  
  
    public Animal() {  
  
    }  
  
    public Animal(String name, String  
color, int age) {  
  
        this.name = name;
```

```

        this.color = color;

        this.age = age;

    }

    public abstract void eat();

}

%-----%
package shiyan6.exa10;

public class Cat extends Animal {

    public void eat() {

        System.out.println("猫吃：猫粮

");

    }

}

%-----%
package shiyan6.exa10;

public class exa {

    public static void main(String[]
args) {

        Animal a1 = new Cat();

        a1.eat();

        Cat a2 = new Cat();

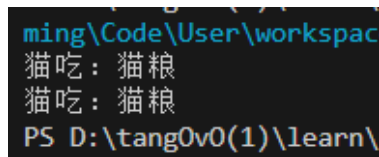
        a2.eat();

    }

```

```
}
```

运行结果：



```
ming\Code\User\workspace
猫吃：猫粮
猫吃：猫粮
PS D:\tangOv0(1)\learn\
```

图11运行结果示意图

12、编写程序，学习接口的使用。

程序：package shiyan6.exa11;

```
/**
```

```
 * 接口：
```

```
 * 1. 什么是接口
```

```
 * 接口是一种更加抽象的类。它完成的是一种特性的功能。
```

```
 * 接口的出现，打破了单继承的局限性。
```

```
 * 2. 接口如何定义及成员特点
```

```
 * public interface 接口名{
```

```
 * //常量
```

```
 * //抽象方法
```

```
 * //默认方法
```

```
 * //静态方法
```

```
 * }
```

```
 * 3. 接口的注意事项
```

```
 * 接口不能有构造方法
```

```
 * 接口不能直接创建对象使用的
```

```
 * 4. 接口与类之间的关系
```

```
 * 实现关系：implements 而且可以多实现
```

```
 * 5. 接口和接口之间的关系
```

```
 * 继承的关系：extends 接口可以多继承
```

```
 */
```

```
interface Vehicle {
```

```
    void start(); // 抽象方法
```

```
    void stop(); // 抽象方法
```

```
}
```

```
// 实现接口的类1
```

```
class Car implements Vehicle {  
    @Override  
    public void start() {  
        System.out.println("汽车启动：踩油门");  
    }  
  
    @Override  
    public void stop() {  
        System.out.println("汽车停止：踩刹车");  
    }  
}
```

```
// 实现接口的类2
```

```
class Bicycle implements Vehicle {  
    @Override  
    public void start() {  
        System.out.println("自行车启动：开始蹬踏板");  
    }  
  
    @Override  
    public void stop() {  
        System.out.println("自行车停止：捏刹车");  
    }  
}
```

```
// 主程序
```

```
public class exa {  
    public static void main(String[] args) {  
        System.out.println("=== 简单的接口示例 ===\n");  
  
        // 创建对象
```

```

Vehicle car = new Car();
Vehicle bike = new Bicycle();

// 使用接口引用调用方法
System.out.println("汽车操作:");
car.start();
car.stop();

System.out.println("\n自行车操作:");
bike.start();
bike.stop();

// 演示多态
System.out.println("\n=== 多态演示 ===");
operateVehicle(car);
operateVehicle(bike);
}

// 使用接口类型作为参数 - 多态
public static void operateVehicle(Vehicle v) {
    System.out.println("\n操作交通工具:");
    v.start();
    v.stop();
}
}

```

运行结果：

```
=== 简单的接口示例 ===

汽车操作：
汽车启动：踩油门
汽车停止：踩刹车

自行车操作：
自行车启动：开始蹬踏板
自行车停止：捏刹车

=== 多态演示 ===
=== 多态演示 ===

操作交通工具：
汽车启动：踩油门
汽车停止：踩刹车

操作交通工具：
自行车启动：开始蹬踏板
自行车停止：捏刹车
```

图12运行结果示意图

13、编写程序，学习 Java 程序中的多态。

程序：package shiyan6.exa12;

/*

* 多态

* 1. 什么是多态

* 指的是事物的多种表现形态。

* 例如：猫是一只动物。猫是一只猫

* 2. 多态的前提条件

* 要有继承或者实现的关系

* 要有方法重写

* 要有父类引用指向子类的对象

* 3. 多态的成员访问特点

* 成员变量：编译看父类、运行看父类。

* 成员方法：编译看父类、运行看子类。

* 4. 多态的好处和弊端

* 好处：提高代码扩展性和维护性

* 弊端：无法使用子类特有的成员

* 5. 多态的使用场景

* 可以作为方法的参数和返回值进行使用。

* 可以提高代码的扩展性。

*/

// 父类

```
class Animal {  
    public void makeSound() {  
        System.out.println("动物发出声音");  
    }  
}
```

// 子类1

```
class Dog extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("汪汪！");  
    }  
}
```

// 子类2

```
class Cat extends Animal {  
    @Override  
    public void makeSound() {  
        System.out.println("喵喵！");  
    }  
}
```

// 主程序

```
public class exa {  
    public static void main(String[] args) {  
        System.out.println("=== 多态演示 ===\n");  
  
        // 多态：父类引用指向子类对象  
        Animal animal1 = new Dog();    // 向上转型  
        Animal animal2 = new Cat();    // 向上转型
```



```

// 同样的方法调用，产生不同的行为
System.out.println("动物1的叫声：");
animal1.makeSound();

System.out.println("\n动物2的叫声：");
animal2.makeSound();

// 使用数组演示多态
System.out.println("\n=== 数组多态演示 ===");
Animal[] animals = {new Dog(), new Cat(), new Dog()};

for (Animal animal : animals) {
    animal.makeSound();    // 运行时确定调用哪个方法
}

// 方法参数多态
System.out.println("\n=== 方法参数多态 ===");
playWithAnimal(new Dog());
playWithAnimal(new Cat());
}

// 方法参数使用父类类型，可以接受任何子类对象
public static void playWithAnimal(Animal animal) {
    System.out.print("和动物玩耍，它说：");
    animal.makeSound();
}
}

```

运行结果：

```
=== 多态演示 ===

动物1的叫声：
汪汪！

动物2的叫声：
喵喵！

=== 数组多态演示 ===
汪汪！
喵喵！
汪汪！

=== 方法参数多态 ===
和动物玩耍，它说：汪汪！
和动物玩耍，它说：喵喵！
```

图13 运行结果示意图

14、编写程序，学习多态的转型，关键字 instanceof 的使用。

```
程序： package shiyan6.exa13;

/**
 * 1. 多态的转型
 * 向上转型：父类引用指向子类对象
 * 向下转型：由父类引用转成一个对应的真实的子类对象
 * 格式：目标对象类型 对象名 = （目标对象类型）被转换的引用
 * 2. 注意事项：
 * 一定要确保转换的类型相同。否则会发类型转换异常：ClassCastException
 * 3. 关键字：
 * instanceof 用于判断左边的引用是否是右边的对象类型
 */

// 父类
class Animal {
    public void eat() {
        System.out.println("动物吃东西");
    }
}

// 子类
```

```
class Dog extends Animal {
    @Override
    public void eat() {
        System.out.println("狗吃骨头");
    }

    // 狗特有的方法
    public void bark() {
        System.out.println("汪汪叫");
    }
}

class Cat extends Animal {
    @Override
    public void eat() {
        System.out.println("猫吃鱼");
    }

    // 猫特有的方法
    public void meow() {
        System.out.println("喵喵叫");
    }
}

public class exa{
    public static void main(String[] args) {
        // 1. 向上转型 - 多态
        Animal animal1 = new Dog();    // 子类转父类
        Animal animal2 = new Cat();    // 子类转父类

        System.out.println("=== 多态演示 ===");
        animal1.eat();    // 输出：狗吃骨头
        animal2.eat();    // 输出：猫吃鱼
    }
}
```

```

// 2. instanceof 使用
System.out.println("\n=== instanceof 演示 ===");
System.out.println("animal1 是 Dog: " + (animal1 instanceof
Dog)); // true
System.out.println("animal1 是 Cat: " + (animal1 instanceof
Cat)); // false
System.out.println("animal1 是 Animal: " + (animal1 instanceof
Animal)); // true

```

```

// 3. 向下转型
System.out.println("\n=== 向下转型演示 ===");
if (animal1 instanceof Dog) {
    Dog dog = (Dog) animal1; // 父类转子类
    dog.bark(); // 调用狗特有的方法
}

```

```

if (animal2 instanceof Cat) {
    Cat cat = (Cat) animal2; // 父类转子类
    cat.meow(); // 调用猫特有的方法
}

```

```

// 4. 错误的转型示例
System.out.println("\n=== 错误转型处理 ===");
Animal animal = new Dog();

// 错误方式：直接转型会报错
// Cat wrongCat = (Cat) animal; // 运行时报错：
ClassCastException

```

```

// 正确方式：先用instanceof检查
if (animal instanceof Cat) {
    Cat cat = (Cat) animal;
    cat.meow();
} else {
    System.out.println("不能将动物转换为猫");
}

```

```

    }

    // 5. 数组中的多态
    System.out.println("\n=== 数组多态演示 ===");
    Animal[] animals = {new Dog(), new Cat(), new Dog()};

    for (Animal a : animals) {
        a.eat();    // 多态调用

        if (a instanceof Dog) {
            ((Dog) a).bark();    // 直接转型调用
        } else if (a instanceof Cat) {
            ((Cat) a).meow();    // 直接转型调用
        }
    }
}
}
}

```

运行结果：

```

=== 多态演示 ===
狗吃骨头
猫吃鱼

=== instanceof 演示 ===
animal1 是 Dog: true
animal1 是 Cat: false
animal1 是 Animal: true

=== 向下转型演示 ===
汪汪叫
喵喵叫

=== 错误转型处理 ===
不能将动物转换为猫

=== 数组多态演示 ===
狗吃骨头
汪汪叫
猫吃鱼
喵喵叫
狗吃骨头
汪汪叫

```

图14运行结果示意图

15、编写程序，学习如何定义成员内部类以及如何在外部类中访问内部类。

程序：package shiyan6.exa14;

/**

* 创建内部类对象的格式：

* 外部类.内部类 对象名 = new 外部类().new 内部类()

* 成员内部类可以访问外部类的所有成员，无论外部类的成员是何种访问权限。

*/

// 简单版外部类

```
class University {
    private String universityName;

    public University(String name) {
        this.universityName = name;
    }

    // 成员内部类 - 学院
    class College {
        private String collegeName;

        public College(String name) {
            this.collegeName = name;
        }

        public void showInfo() {
            System.out.println(collegeName + "学院属于" +
universityName);
        }

        public void accessUniversity() {
            System.out.println("内部类可以访问外部类的名字: " +
universityName);
        }
    }

    // 外部类方法创建内部类
    public College createCollege(String name) {
        return new College(name);
    }

    // 外部类访问内部类
```

```

        public void manageCollege() {
            College computerCollege = new College("计算机");
            computerCollege.showInfo();
        }
    }

    public class exa {
        public static void main(String[] args) {
            // 创建外部类
            University university = new University("清华大学");

            // 创建内部类方式1: 通过外部类实例
            University.College college1 = university.new College("物理");
            college1.showInfo();
            college1.accessUniversity();

            // 创建内部类方式2: 通过外部类方法
            University.College college2 = university.createCollege("数学");
            college2.showInfo();

            // 外部类访问内部类
            university.manageCollege();

            // 链式创建
            University.College college3 = new University("北京大学").new
            College("化学");
            college3.showInfo();
        }
    }
}

```

运行结果：

```

物理学院属于清华大学
内部类可以访问外部类的名字：清华大学
数学学院属于清华大学
计算机学院属于清华大学
化学学院属于北京大学
PS D:\tang0v0(1)\learn\java>
化学学院属于北京大学
化学学院属于北京大学
PS D:\tang0v0(1)\learn\java>

```

图15运行结果示意图

16、编写程序，局部内部类的定义以及如何访问局部内部类。

程序：package shiyan6.exa15;

/**

* 局部内部类可以访问外部类所有成员，而在外部类中无法直接访问局部内部类中的成员

* 如何访问局部内部类？

* 只需创建外部类对象。调用所属方法即可。

*/

public class exa {

public static void main(String[] args) {

// 直接在main方法中使用局部内部类

class Greeter {

public void greet(String name) {

System.out.println("你好, " + name);

}

}

Greeter greeter = new Greeter();

greeter.greet("张三");

greeter.greet("李四");

// 另一个例子

testLocalClass();

}

static void testLocalClass() {

final int base = 10;

class Adder {

public void add(int x) {

System.out.println(base + " + " + x + " = "

+ (base + x));

}


```

    }

    Adder adder = new Adder();

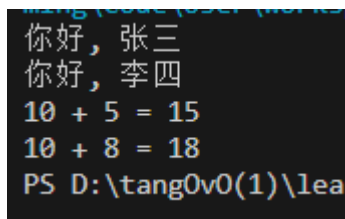
    adder.add(5);

    adder.add(8);

}
}

```

运行结果：



```

你好, 张三
你好, 李四
10 + 5 = 15
10 + 8 = 18
PS D:\tang0v0(1)\lea

```

图16运行结果示意图

17、编写程序，学习静态内部类的定义和使用。

程序：package shiyan6.exa16;

/**

* 静态成员内部类：

* 静态内部类只能访问外部类的静态成员

* 创建对象格式：

* 外部类名.内部类名 对象名 = new Outer.Inner();

* 外部类名.内部类名 对象名 = new Outer().new Inner();(普通成员内部类)

*/

```
public class exa {
```

```
    // 外部类
```

```
    static class University {
```

```
        static String name = "清华大学";
```

```
        // 静态内部类
```

```
        static class College {
```

```
            String collegeName;
```

```
            College(String name) {
```

```
                this.collegeName = name;
```

```

    }

    void showInfo() {
        System.out.println(collegeName + "学院 - " +
University.name);
    }

    static void universityInfo() {
        System.out.println("大学名称: " + name);
    }
}

public static void main(String[] args) {
    // 直接创建静态内部类，不需要外部类实例
    University.College college1 = new University.College("计算机");
    University.College college2 = new University.College("物理");

    college1.showInfo();
    college2.showInfo();

    // 调用静态内部类的静态方法
    University.College.universityInfo();

    // 另一个例子：计算器
    Calculator.Operation add = new Calculator.Operation("加法");
    add.execute(5, 3);

    Calculator.Operation.multiply(4, 6);
}

// 计算器类
class Calculator {

```

```

// 静态内部类：操作
static class Operation {
    String name;

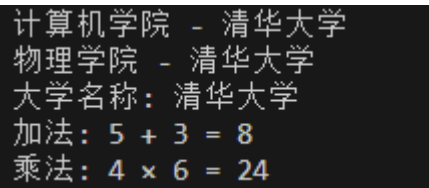
    Operation(String name) {
        this.name = name;
    }

    void execute(int a, int b) {
        System.out.println(name + ": " + a + " + " + b +
" = " + (a + b));
    }

    static void multiply(int a, int b) {
        System.out.println("乘法: " + a + " × " + b + "
= " + (a * b));
    }
}

```

运行结果：



```

计算机学院 - 清华大学
物理学院 - 清华大学
大学名称: 清华大学
加法: 5 + 3 = 8
乘法: 4 x 6 = 24

```

图17运行结果示意图

18、编写程序，学习匿名内部类的定义和使用。

程序： package shiyan6.exa17;

```

public abstract class Animal {
    public abstract void eat(); // 传统的方式必须是子类重写
}

%-----%
package shiyan6.exa17;

import shiyan6.exa03.Animal;

```

```

public class Dog extends Animal {
    public void eat() {
        System.out.println("狗吃肉！");
    }
}

%-----%
package shiyan6.exa17;

import shiyan6.exa03.Animal;

/**
 * 匿名内部类的前提：必须是类或者接口
 * 格式：
 * new 类名/接口名() { //子类或接口的实现类
 * 重写抽象方法
 * }
 * 用途：作为方法的参数传递（前提是类或接口只有一个方法的时候），
 * 多个方法时，建议建立一个类实现接口或继承父类的形式。
 * 传统实现方式：
 * 1. 编写实现类
 * 2. 重写抽象方法
 * 3. 创建实现类对象
 * 4. 将实现类对象作为方法的参数传递
 */
public class exa {
    public static void main(String[] args) {
        /*
         * 传统的方式必须是子类重写，创建子类对象调用方法
         * Dog d = new Dog();
         * d.eat();
         */
        useAnimal(new Animal() {
            public void eat() {

```

```

        System.out.println("狗吃骨头----!");
    }

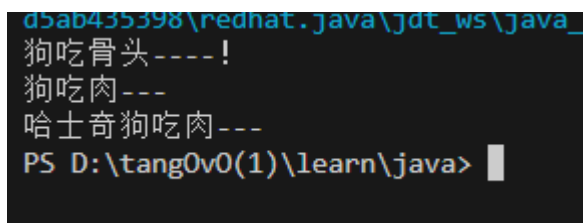
});
// 调用方式一
// 整体就等效于：是 Animal 父类的子类对象
new Animal() {
    @Override
    public void eat() {
        System.out.println("狗吃肉---");
    }
}.eat();
// 调用方式二
// 通过匿名内部类访问局部变量。在 JDK8 版本之前，必须加 final
关键字

String name = "哈士奇";
// name = "金毛";//不能赋值
Animal a = new Animal() {
    @Override
    public void eat() {
        System.out.println(name + "狗吃肉---");
    }
};
a.eat();
}

public static void useAnimal(Animal a) {
    a.eat();
}
}

```

运行结果：



```

d5ab435398\redhat.java\jdt_ws\java_
狗吃骨头----!
狗吃肉---
哈士奇狗吃肉---
PS D:\tangOv0(1)\learn\java>

```

图18运行结果示意图

19、控制台程序的输入输出处理。

编写java应用程序，完成从键盘输入两个运算数据，计算两数之和并输出结果的功能。

程序1: package shiyan6.exa18;

```
/**
 * 根据运行界面是以图形化要素为主，还是以文本字符为主，可分为两种主要类型：
 * GUI 应用程序(GUI Application)
 * 控制台应用程序(Console Application)
 */
import java.io.*; // 以便在程序中调用相关的类和方法

public class MyDemo01 {
    // 输入输出异常，由虚拟机自行处理
    public static void main(String args[]) throws IOException {
        // 输入数据
        // 控制台输入方案
        System.out.println("输入一个整数：");
        byte t[] = new byte[10];
        System.in.read(t); // 用于获取用户从键盘输入的数据。
        String s1 = new String(t);
        int a = Integer.parseInt(s1.trim()); // 去除空格
        System.out.println("输入一个小数：");
        System.in.read(t);
        String s2 = new String(t);
        double b = Double.parseDouble(s2.trim());
        // 处理数据
        double c = a + b;
        // 输出结果
        // 控制台输出方案
        System.out.println("计算的结果为：" + c);
    }
}
```

```

%-----%
package shiyan6.exa18;

import java.util.Scanner;

/**
 * Scanner scan = new Scanner(System.in);
 * 是 Java 中用于从标准输入（通常是键盘）读取用户输入的代码。
 * 它的主要功能如下：
 * 创建 Scanner 对象：
 * new Scanner(System.in)创建了一个 Scanner 类的实例，用于处理输入流。
 * 指定输入源：
 * System.in 表示标准输入流，通常对应键盘输入，这意味着 Scanner 将从键盘读取用户输入的数据。
 * 提供多种输入方法：
 * 通过创建的 scan 对象，可以使用 Scanner 类提供的一系列方法读取不同类型的输入。
 * 例如：
 * nextInt()：读取整数
 * nextDouble()：读取双精度浮点数
 * nextLine()：读取一行文本
 * next()：读取一个单词（以空格为分隔符）
 * 使用完毕后，建议调用 scan.close()关闭 Scanner，以释放相关资源。
 */
public class MyDemo02 {
    /* 两数求和 */
    public static void main(String args[]) {
        try (// 输入数据
            // 控制台输入方案
            Scanner scan = new Scanner(System.in)) {
            System.out.println("输入一个整数：");
            int a = scan.nextInt();
            System.out.println("输入一个小数：");
            double b = scan.nextDouble();
            // 处理数据

```

```

        double c = a + b;

        // 输出结果
        // 控制台输出方案
        System.out.println("计算的结果为：" + c);

        // 使用完毕后，建议调用 scan.close() 关闭 Scanner，以释放
        相关资源。
    }

}
}
}

```

运行结果：

```

at.java\jdt_ws\java_b497f865\
输入一个整数:
12
输入一个小数:
0.12
计算的结果为: 12.12

```

图19 运行结果示意图

20、图形化程序的输入输出处理

编写Java程序，利用图形化界面，完成从键盘输入两个运算数据，计算两数之和并输出结果的功能。

程序：package shiyan6.exa18;

```
import javax.swing.JOptionPane;
```

```
/**
```

- * 1. javax.swing 包中包含很多创建 Java 图形用户界面应用程序所必需的类。
- * 将 JOptionPane 类引入当前程序，方便在程序中调用相关方法，实现相关输入输出功能。
- * 2. 调用类 JOptionPane 的 showInputDialog 方法显示“输入”对话框。
- * showInputDialog 方法的参数为提示信息，用以提示用户输入相关内容。
- * 在文本框中输入相关字符信息后，单击“确定”按钮或按 Enter 键可以把文本框中的字符信息返回给 Java 程序。
- * 3. 调用类 JOptionPane 的 showMessageDialog 方法，打开“消息”对话框显示结果信息。
- * 这个方法包含两个参数，参数之间用逗号分隔。
- * 第一个参数表示对话框的父窗口对象，当使用关键字 null 时，表示对话框的父窗口不存在，对话框将直接显示在计算机的显示器屏幕上；


```
* 第二个参数为对话框中要显示的信息，类型为字符串。
*/
public class MyDemo03 {
    public static void main(String[] args) {
        String s1 = JOptionPane.showInputDialog("输入一个整数");
        int a = Integer.parseInt(s1);
        String s2 = JOptionPane.showInputDialog("输入一个小数");
        double b = Double.parseDouble(s2);
        // 处理数据
        double c = a + b;
        // 输出结果
        // 图形化输出方案
        JOptionPane.showMessageDialog(null, "结果为: " + a + "+" + b
+ "=" + c);
    }
}
```

运行结果：

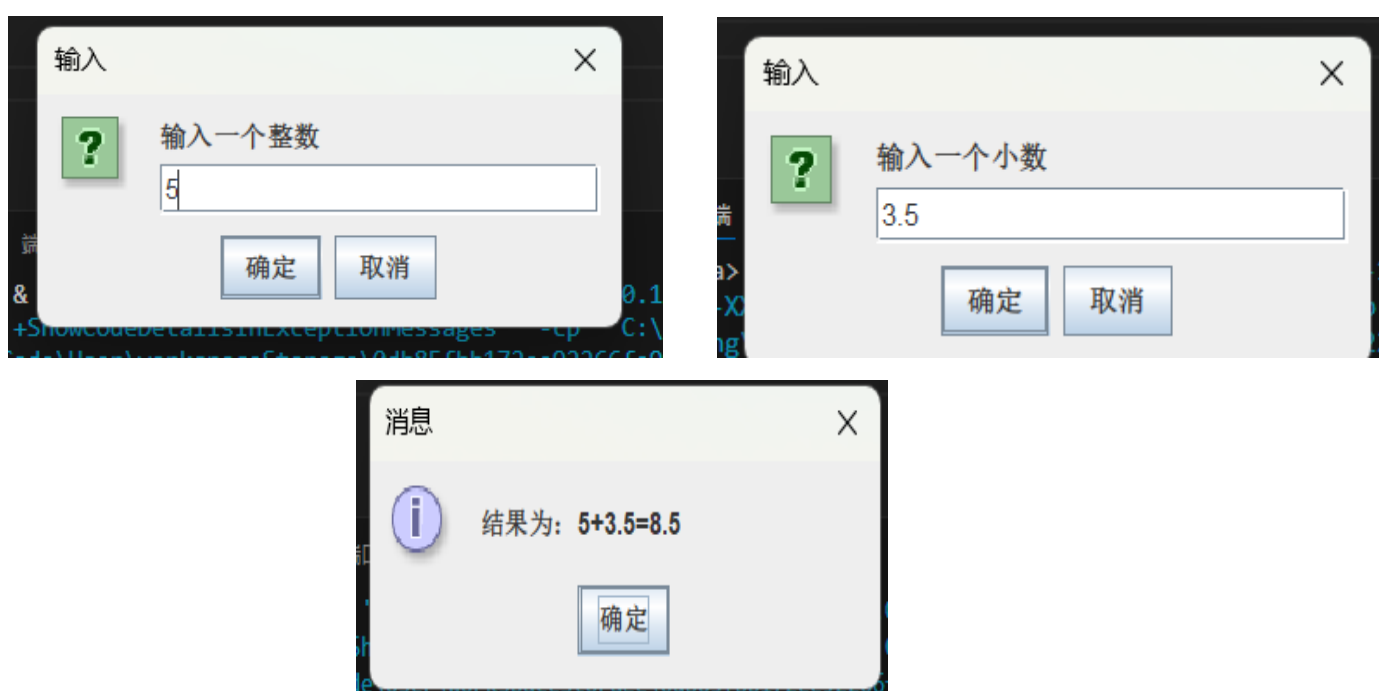


图20运行结果示意图

21、计算圆的周长和面积

(1) 编写Java控制台应用程序，完成从键盘输入圆的半径，计算圆的周长和面积并输出结果的功能。

程序:

```
package shiyan6.exa19;

import java.io.* ;

public class Circle {

    /*键盘输入半径，求圆的周长和面积*/

    public static void main(String[] args) throws IOException{

        // 定义常量PI

        final double PI = 3.14;

        // 定义变量

        byte buf[] = new byte[50] ;

        double r,girth,area;

        // 输入圆的半径

        System.out.println("请输入一个圆的半径：");

        System.in.read(buf) ;

        String str = new String(buf) ;

        r = Double.parseDouble(str.trim()) ;

        // 计算周长和面积

        girth = 2 * PI * r;

        area = PI * r * r;

        // 输出结果

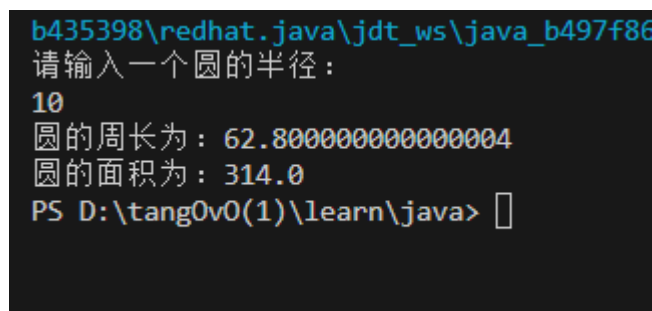
        System.out.println("圆的周长为：" + girth);

        System.out.println("圆的面积为：" + area);

    }

}
```

运行结果:



```
b435398\redhat.java\jdt_ws\java_b497f86
请输入一个圆的半径：
10
圆的周长为： 62.800000000000004
圆的面积为： 314.0
PS D:\tang0v0(1)\learn\java> 
```

图 1.20 运行结果示意图

%------%

(2) 编写Java GUI应用程序，完成从键盘输入矩形的长和宽，求矩形的周长和面积并输出结果的功能。

程序: package shiyan6.exa19;

import javax.swing.JOptionPane;

```
public class exc {  
    /* 键盘输入长和宽，计算矩形的周长和面积 */  
    public static void main(String args[]) {  
        // 定义变量  
        String s;  
        double length, width, girth, area;  
        // 输入矩形的长和宽  
        s = JOptionPane.showInputDialog("输入矩形的长: ");  
        length = Double.parseDouble(s);  
        s = JOptionPane.showInputDialog("输入矩形的宽: ");  
        width = Double.parseDouble(s);  
        // 计算周长和面积  
        girth = (length + width) * 2;  
        area = length * width;  
        // 输出计算结果  
        JOptionPane.showMessageDialog(null, "周长为: " + girth + "\n"  
+ "面积为: "  
        + area);  
    }  
}
```

运行结果:

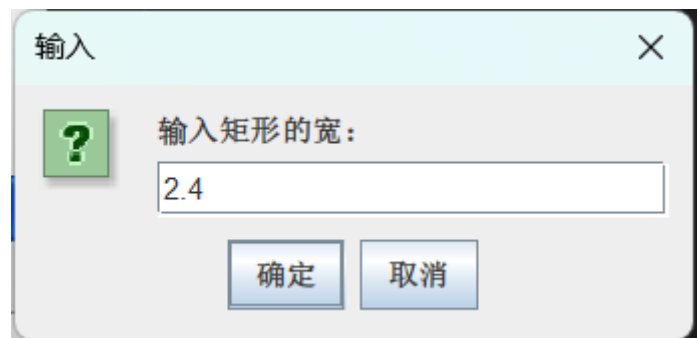
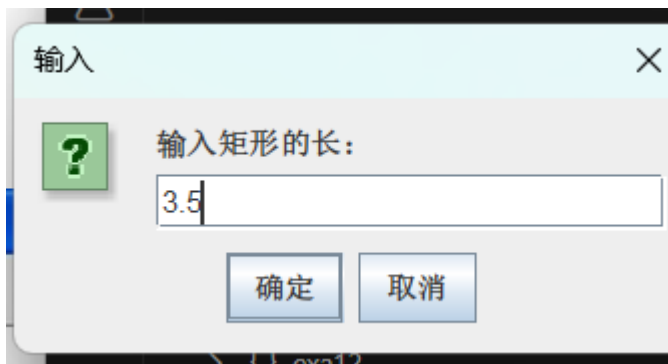




图 1.21 运行结果示意图

22、超市计价器的实现。

小明在超市购买了一瓶饮料和一个面包，请编写一个图形化的超市计价器程序，帮助他计算一下商品的总价格。

```
程序: package shiyan6.exa20;

import javax.swing.JOptionPane;

public class Calculator {
    public static void main(String[] args) {
        // 定义所需变量
        String temp;
        double oneprice, twoprice, totalprice;
        // 输入两种商品的价格
        temp = JOptionPane.showInputDialog("请输入饮料的价格");
        oneprice = Double.parseDouble(temp);
        temp = JOptionPane.showInputDialog("请输入面包的价格: ");
        twoprice = Double.parseDouble(temp);
        // 计算两种商品的总价格
        totalprice = oneprice + twoprice;
        // 输出计算的总价格
        JOptionPane.showMessageDialog(null,
            "饮料价格: " + oneprice + "元\n" + "面包价格: "
            + twoprice + "元\n" + "商品总价格: " + totalprice + "元");
    }
}
```

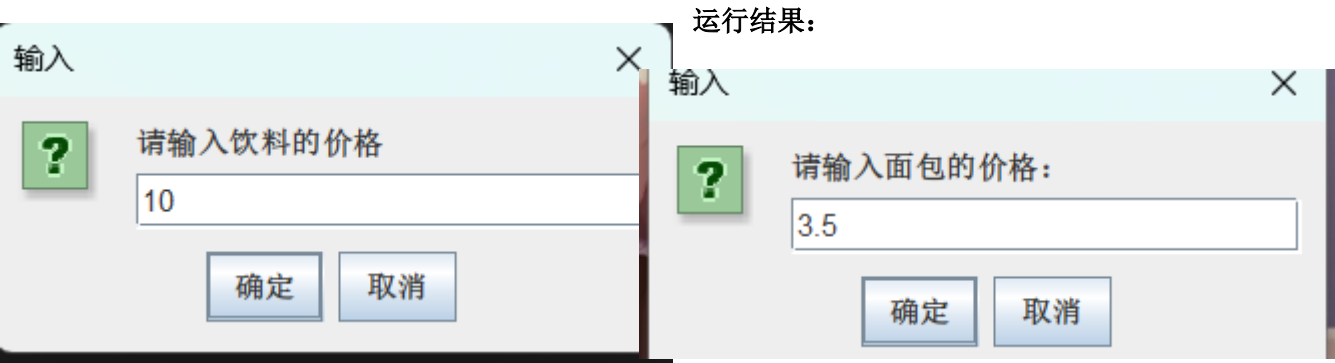




图 1.22 运行结果示意图

三 、上机体会

本次上机实验在老师的带领下实打实的使用private进行封装，以及如何调用封装的对象，并且在实验中也明白了如何对数据进行保护，也进行了如何重载的实验，对构造方法进行了多次重载并且也多次尝试不同的测验。后面我们对this关键字进行使用，进一步理解this的使用过程和目的，以及static关键字在修饰成员变量和修饰方法中的使用。以及对继承和重写的再一次练习，还学习重写toString方法，super关键字的使用，final关键字的使用，抽象类，接口和多态的使用，以及进行了与现实相关的交互实验。

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 异常

实验时间 2025 年 10 月 10.17 日 第 34 节

年级 2024 专业班级 计科1班

姓名 唐明强 学 号 20242122002

评分 [单击此处成绩](#)

王庆平

教师签名

上机二 面向对象基础

一、上机目的

- 1、了解什么是运行时异常和编译时异常，能够说出运行时异常和编译时异常的特点。
- 2、了解异常的产生及处理，能够说出处理异常的5个关键字。
- 3、掌握能够使用try...catch语句和finally语句处理异常。
- 4、掌握throws关键字的使用，能够使用throws关键字抛出异常。
- 5、掌握throw关键字的使用，能够使用throw关键字抛出异常。
- 6、掌握如何自定义异常，能够编写自定义异常类。

二、上机任务

- 1、编写程序，计算以0为除数的表达式，运行程序并观察程序的运行结果。

程序: package com.company.computer01;

/**

- * 异常指程序运行过程中出现的非正常现象:
 - *
 - * 例如文件找不到、用户输入错误、除数为零、数组下标越界等。
 - * 异常是一个事件，它发生在程序运行期间，干扰了正常的指令流程。
 - * Java 通过API中Throwable类的众多子类描述各种不同的异常。
 - * Java 异常都是对象，是Throwable子类的实例，描述了出现在一段编码中的错误条件。
 - * 当条件生成时，错误将引发异常。
 - * Throwable: 有两个重要的子类:
 - *
 - * Error (错误): 是指程序无法处理的错误，表示运行应用程序中较严重问题。
 - *
 - *
 - * Exception (异常): 是程序本身可以处理的异常，常分两大类:
 - * 运行时异常和非运行时异常(编译异常)。程序中应当尽可能去处理这些异常。
- */

```
public class Example01 {  
    public static void main(String[] args) {  
        int result = divide(4, 0);  
        // 调用divide()方法，第2个参数为0  
        System.out.println(result);  
    }  
}
```

// 下面的方法实现了两个整数相除

```
public static int divide(int x, int y) {  
    int result = x / y; // 定义一个变量result记录两个数相除的结果  
    return result;  
    // 将结果返回  
}  
}
```

运行结果:

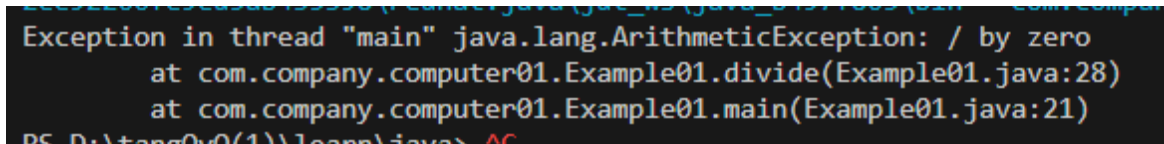


图 2.1 运行结果示意图

2、编写程序，使用try...catch语句对计算以0为除数的表达式出现的异常进行捕获。

程序: package com.company.computer02;

```
/**  
 * 异常处理，就是指程序在出现问题时依然可以正确的执行完。  
 * 在Java应用程序中，异常处理机制分为：  
 * 1) 抛出异常  
 * 当一个方法出现错误引发异常时，方法创建异常对象并交付运行时系统；  
 * 异常对象中包含了异常类型和异常出现时的程序状态等异常信息。  
 * 运行时系统负责寻找处置异常的代码并执行。  
 * 2) 捕捉异常。  
 * 在方法抛出异常之后，运行时系统将转为寻找合适的异常处理器（exceptionhandler）。  
 * 潜在的异常处理器是异常发生时依次存留在调用栈中的方法的集合。  
 * 通过try-catch 语句捕获异常，这两个语句必须同时使用。其一般语法形式为：  
 * try {  
 * // 可能会发生异常的程序代码  
 * } catch （异常类型 异常的变量名1){  
 * // 捕获并处置try抛出的异常类型1  
 * } catch （异常类型 异常的变量名2){  
 * // 捕获并处置try抛出的异常类型2  
 * }  
 */  
  
public class Example02 {  
    public static void main(String[] args) {  
        try { // try 监控区域
```



```

        int result = divide(4, 0); // 调用 divide()方法, 第2
个参数为0

        System.out.println(result);
    } catch (ArithmeticException e) {
        System.out.println(e.toString());
    }
    System.out.println("程序继续会执行下去-----");
}

// 下面的方法实现了两个整数相除
public static int divide(int x, int y) {
    int result = x / y; // 定义一个变量result记录两个数相除的结果
    return result;      // 将结果返回
}
}

```

运行结果:

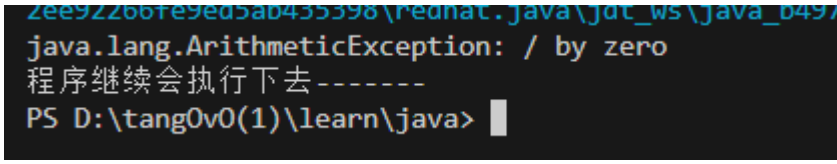


图 2.2 运行结果示意图

3、编写程序，演示try...catch...finally语句块的使用。

程序: package com.company.computer03;

```

/**
 * try-catch-finally 语句:
 * try 块: 用于捕获异常。
 * 其后可接零个或多个catch块, 如果没有catch块, 则必须跟一个finally块。
 * catch 块: 用于处理try捕获到的异常。
 * finally 块: 无论是否捕获或处理异常, finally块里的语句都会被执行。
 * 当在try块或catch块中遇到return语句时, finally语句块将在方法返回之前被执行。
 * try-catch-finally 语句的一般语法形式为:
 * try{
 * // 可能会发生异常的程序代码
 * }catch(Type1 id1){
 * // 捕获并处理try抛出的异常类型Type1
 * }catch(Type2 id2){

```

```

* // 捕获并处理try抛出的异常类型Type2
* }finally{
* // 无论是否发生异常，都将执行的语句块
* }
* 在以下4种特殊情况下，finally块不会被执行：
* 1) 在finally语句块中发生了异常；
* 2) 在前面的代码中用了System.exit()退出程序。
* 3) 程序所在的线程死亡。
* 4) 关闭CPU。
* try、catch、finally 语句块的执行顺序：
* 1) 当try没有捕获到异常时：try语句块中的语句逐一被执行，程序将跳过catch语句
* 块，执行finally语句块和其后的语句；
* 2) 当try语句块里的某条语句出现异常时，而没有处理此异常的catch语句块时，此异
* 常将会抛给JVM处理，finally语句块里的语句还是会被执行，但finally语句块后的语
* 句不会被执行；
* 3) 当try捕获到异常，catch语句块里有处理此异常的情况：在try语句块中是按照顺
* 序来执行的，当执行到某一条语句出现异常时，程序将跳到catch语句块，并与catch
* 语句块逐一匹配，找到与之对应的处理程序，其他的catch语句块将不会被执行，而try
* 语句块中，出现异常之后的语句也不会被执行，catch语句块执行完后，执行finally语
* 句块里的语句，最后执行finally语句块后的语句。
*/
public class Example03 {
    public static void main(String[] args) {
        try {
            int result = divide(4, 0); // 调用 divide()方法，第
2个参数为0

            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println(e.toString());
            return;
        } finally {
            System.out.println("程序继续会执行下去-----");
        }
    }
}

// 下面的方法实现了两个整数相除

```

```

public static int divide(int x, int y) {
    int result = x / y; // 定义一个变量result记录两个数相除的结果
    return result;
}
// 将结果返回
}

```

运行结果:

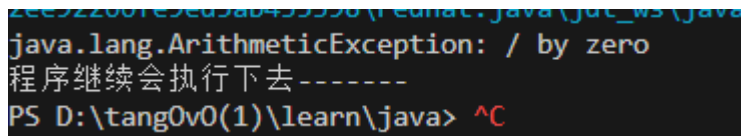


图 2.3 运行结果示意图

4、编写程序，演示throws关键字的使用。

程序: package com.company.computer04;

```

/**
 * 方法体
 * 修饰符 返回值类型 方法名(参数1, 参数2, ...)throws 异常类1, 异常类2... {
 *
 * }
 * throws 关键字声明该方法有可能发生的异常，这样调用者在调用方法时，
 * 就明确地知道该方法有异常，并且必须在程序中对异常进行处理，否则编译无法通过。
 */
public class Example04 {
    public static void main(String[] args) {
        // 下面的代码定义了一个try...catch语句用于捕获异常
        try {
            int result = divide(4, 2); // 调用divide()方法
            System.out.println("4÷2=" + result);
        } catch (Exception e) {
            e.printStackTrace();
            // 对捕获到的异常进行处理
            // 打印捕获的异常信息
        }
    }

    // 实现了两个整数相除，并使用throws关键字声明抛出异常
    public static int divide(int x, int y) throws Exception {

```

```

        int result = x / y; // 定义变量result记录两个数相除的结果
        return result;
    } // 将结果返回
}

```

运行结果：

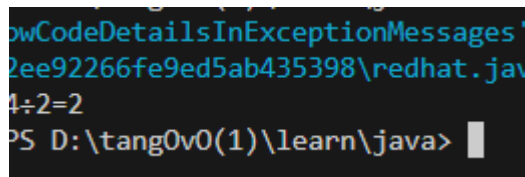


图 2.4 运行结果示意图

5. 编写程序，演示throws关键字的使用。

程序：package com.company.computer05;

```

/**
 * throw 总是出现在函数体中，用来抛出一个Throwable类型的异常。
 * 程序会在throw语句后立即终止，它后面的语句执行不到，
 * 然后在包含它的所有try块中（可能在上层调用函数中）从里向外寻找含有与其匹配的
 * catch 子句的try块。
 */

public class Example05 {
    public static void main(String[] args) {
        // 下面的代码定义了一个try...catch语句用于捕获异常
        int age = -1;
        try {
            printAge(age);
        } catch (Exception e) { // 对捕获到的异常进行处理
            System.out.println("捕获的异常信息为：" + e.getMessage());
        }
    }

    public static void printAge(int age) throws Exception {
        if (age <= 0) {
            // 对业务逻辑进行判断，当输入年龄为负数时抛出异常
            throw new Exception("输入的年龄有误，必须是正整数！");
        } else {

```

```

        System.out.println("此人年龄为: " + age);
    }
}
}

```

运行结果:

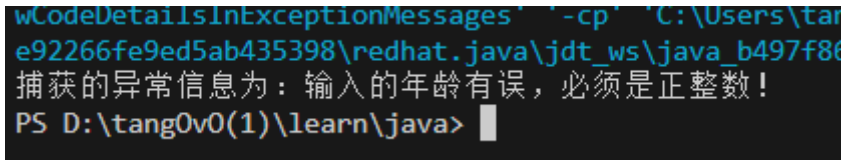


图 2.5 运行结果示意图

6、编写程序，自定义的异常，在divide()方法中判断被除数是否为负数，如果为负数，就使用throw关键字在方法中向调用者抛出自定义的DivideByMinusException异常对象。

程序: package com.company.computer06;

```

/**
 * 自定义的异常类只需继承Exception类，在构造方法中使用super()语句调用Exception
 * 的构造方法即可。
 */
public class DivideByMinusException extends Exception {
    public DivideByMinusException() {
        super();
    }

    public DivideByMinusException(String message) {
        super(message);
    }
}

%-----%

package com.company.computer06;

public class Example06 {
    public static void main(String[] args) {
        try {
            System.out.println(divide(-4, 2));
        } catch (DivideByMinusException e) {
            System.out.println(e.toString());
        }
    }
}

```

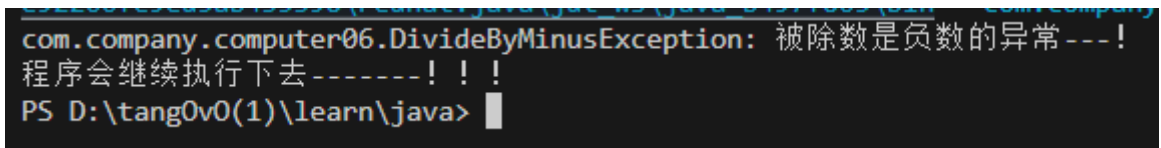
```

        System.out.println("程序会继续执行下去-----!!!");
    }

    public static int divide(int x, int y) throws DivideByMinusException {
        if (x < 0)
            throw new DivideByMinusException("被除数是负数的异常---!");
        else {
            int result = x / y;
            return result;
        }
    }
}

```

运行结果:



```

com.company.computer06.DivideByMinusException: 被除数是负数的异常---!
程序会继续执行下去-----!!!
PS D:\tang0v0(1)\learn\java>

```

图 2.6 运行结果示意图

三、上机体会

本次实验了解了运行时异常和编译时异常，初步掌握了try...catch, 语句和finally语句处理异常，初步掌握throws关键字的使用和能够使用throws关键字抛出异常，初步掌握throw关键字的使用和能够使用throw关键字抛出异常，还有对于通过使用dicide方法来断条件与throw关键字的异常来自定义异常。

滇池学院理工学院

实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 集合

实验时间 2025 年 10 月 24.31 日 第 34 节

年级 2024 专业班级 计科1班

姓名 唐明强 学 号 20242122002

评分 [单击此处成绩](#)

王庆平

教师签名

上机三 集合

一、上机目的

- 1、了解集合的概念,能够说出集合用于做什么。
- 2、熟悉Collection接口,能够说出Collection接口中的常用方法。
- 3、掌握List接口、Set接口、Map接口的使用。
- 4、掌握List接口中的ArrayList、LinkedList、Iterator 接口和foreach循环的使用。
- 5、掌握Set接口中的HashSet、LinkedHashSet和TreeSet的使用。

二、上机任务

- 1、编写程序,演示Collection接口中常用方法的使用。

程序: package com.company.com01;

```
import java.util.*;
```

```
public class CollectionTest {  
    public static void main(String[] args) {  
        // 泛型: 约束集合可以存储数据的类型  
        Collection<String> c1 = new ArrayList<>();  
        // boolean add(Object o) 添加元素  
        c1.add("aa");  
        c1.add("bb");  
        System.out.println(c1);  
        // boolean addAll(Collection c) 将指定集合中所有元素添加到当前集合  
        Collection<String> c2 = new ArrayList<>();  
        c2.add("cc");  
        c2.add("dd");  
        c1.addAll(c2);  
        System.out.println(c1);  
        // boolean contains(Object o) 判断集合是否包含指定元素  
        System.out.println(c1.contains("aa"));  
        System.out.println(c1.contains("aaa"));  
        // boolean containsAll(Collection c) 判断集合中是否包含指定集合所有元素  
        System.out.println(c1.containsAll(c2));  
        // int size() 获取集合的长度  
        System.out.println(c1.size());  
    }  
}
```


}

运行结果：

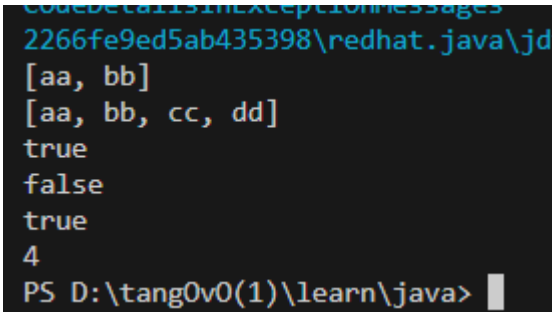


图 3.1 运行结果示意图

2、编写程序，演示list接口中常用方法的使用。

程序：package com.company.com02;

```
import java.util.ArrayList;
import java.util.List;

public class ListDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("bb");
        list.add("cc");
        // void add(int index, Object element) 将指定索引位置添加元素
        list.add(0, "aa");
        System.out.println(list);
        List<String> list2 = new ArrayList<>();
        list2.add("dd");
        list2.add("cc");
        // addAll(int index, Collection c) 将指定集合元素添加到当前集合指定索引
        list.addAll(0, list2);
        System.out.println(list);
        // Object get(int index) 获取指定索引处的元素
        System.out.println(list.get(0));
        // Object remove(int index) 删除指定索引处的元素，返回被删除的元素
        System.out.println(list.remove(0));
        System.out.println(list);
        // Object set(int index, Object obj) 修改指定索引处元素，返回被修改的元
        System.out.println(list.set(0, "qq"));
```

```
        System.out.println(list);
    }
}
```

运行结果：

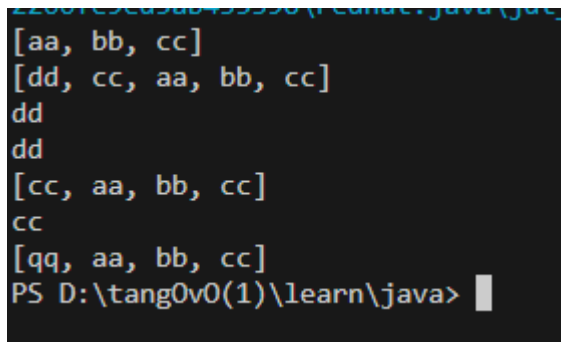


图 3.2 运行结果示意图

3、编写程序，演示ArrayList集合的元素存取。

程序：package com.company.com03;

```
import java.util.ArrayList;
```

```
/**
```

```
 * ArrayList 集合看作一个长度可变的数组。
```

```
 * ArrayList 特点：
```

```
 * 底层是数组结构实现的，数组有索引、查询速度比较快。
```

```
 * 在增加或删除指定位置的元素时，会创建新的数组，增删比较慢。
```

```
 * 使用场景：如果大量的数据经常做查询的操作，优先使用ArrayList
```

```
 */
```

```
public class ArrayListDemo {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> list = new ArrayList<>();
```

```
        // 向集合中添加元素，有序，可重复
```

```
        list.add("张三");
```

```
        list.add("张三");
```

```
        list.add("李四");
```

```
        list.add("王五");
```

```
        list.add("赵六");
```

```
        System.out.println(list);
```

```
        // 获取集合中元素的个数
```

```
        System.out.println("集合的长度：" + list.size());
```

```
        // 取出并打印指定位置的元素
```

```
        System.out.println("第 2 个元素是：" + list.get(1));
```

```

        list.remove(3);
        // 删除索引为3的元素

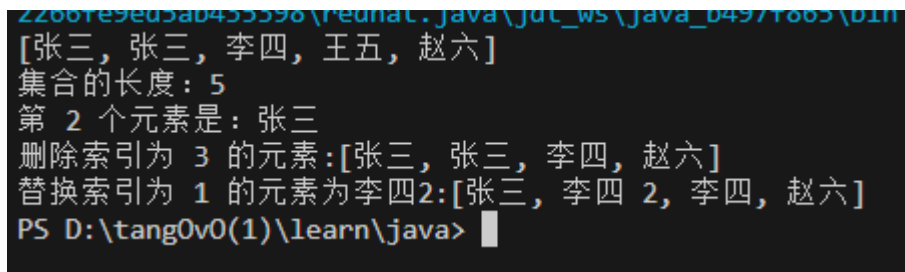
        System.out.println("删除索引为 3 的元素:" + list);

        list.set(1, "李四 2");
        // 替换索引为1的元素为李四2

        System.out.println("替换索引为 1 的元素为李四2:" + list);
    }
}

```

运行结果:



```

2268fe9ed5ad435598\rednat.java\jdk_ws\java_0497f863\bin
[张三, 张三, 李四, 王五, 赵六]
集合的长度: 5
第 2 个元素是: 张三
删除索引为 3 的元素:[张三, 张三, 李四, 赵六]
替换索引为 1 的元素为李四2:[张三, 李四 2, 李四, 赵六]
PS D:\tang0v0(1)\learn\java>

```

图 3.3运行结果示意图

4、编写程序，演示LinkedList集合的元素增加、删除和获取操作。

程序: package com.company.com04;

import java.util.LinkedList;

/**

* LinkedList 集合内部维护了一个双向循环链表，链表中的每一个元素都使用引用的方式记录它的前一个元素和后一个元素，

* 从而可以将所有的元素彼此连接起来。当插入一个新元素时，只需要修改元素之间的引

44

用关系即可，删除一个节点也是如此。

* 正因为这样的存储结构，所以LinkedList集合增删效率非常高。

*/

// 添加元素到开头

public class LinkedListDemo {

public static void main(String[] args) {

LinkedList<String> list = new LinkedList<>();

list.add("张三");

list.add("李四");

list.add("王五");

System.out.println(list);

list.addFirst("赵六");

System.out.println(list);

```
list.addLast("周七");
System.out.println(list);
String first = list.getFirst(); // 获取第一个元素并返回
System.out.println(first);
String last = list.getLast(); // 获取最后一个元素并返回
System.out.println(last);
System.out.println(list.removeFirst()); // 删除第一个元素并返回
System.out.println(list.removeLast()); // 删除最后一个元素并返回
System.out.println(list);
}
}
```

运行结果：

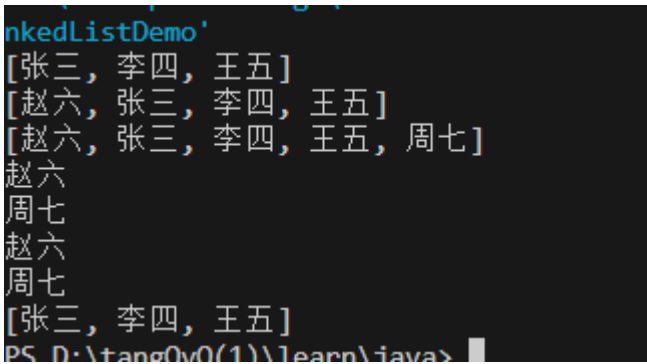


图 3.4运行结果示意图

5、编写程序，演示遍历集合中的元素。

程序：package com.company.com05;

```
import java.util.ArrayList;
import java.util.Iterator;

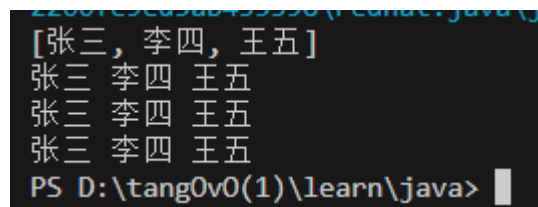
public class IteratorDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("张三");
        list.add("李四");
        list.add("王五");
        // 第1种方法：
        System.out.println(list);
        // 第2种方法：for循环
        for (int i = 0; i < list.size(); i++) {
            // list 集合有索引，可以用for来遍历
```

```

        String s = list.get(i);
        System.out.print(s + " ");
    }
    System.out.println();
    // 第3种方法：增强for循环（foreach循环）
    for (String str : list) {
        System.out.print(str + " ");
        str = "qq";
        // str 是第三方变量，用来依次接收集合中的数据，改变不会影响原集合
        中值
    }
    System.out.println();
    // 第4种方法：迭代器是通用的遍历方式
    Iterator<String> it = list.iterator(); // 1. 获取迭代器对象
    while (it.hasNext()) { // 2. 判断迭代器中是否还有下一个元素
        Object str = it.next(); // 3. 获取迭代器中下一个元素
        System.out.print(str + " ");
    }
}
}

```

运行结果：



```

2200753030043330 (C:\code\java\j
[张三, 李四, 王五]
张三 李四 王五
张三 李四 王五
张三 李四 王五
PS D:\tang0v0(1)\learn\java>

```

图 3.5运行结果示意图

6、编写程序，演示向HashSet集合中添加数据时去除重复数据。

程序1：package com.company.com06;

```

import java.util.HashSet;
import java.util.Iterator;

/**
 * HashSet集合：没有索引、不能存储重复元素、元素唯一存取无序
 * 元素唯一的原理：
 * 1、根据对象的哈希值(调用hashCode() 计算)来计算存储位置

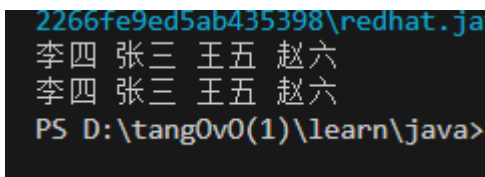
```

```

* 2、判断当前位置时候有数据
* 2.1、没有：直接存储
* 2.2、有：调用equal()方法比较属性值
* 2.2.1、属性值相同：不存储
* 2.2.2、属性值不同：以链表结构存储
**/
public class HashSetDemo01 {
    public static void main(String[] args) {
        HashSet<String> hs = new HashSet<>();
        hs.add("张三");
        hs.add("张三");
        hs.add("李四");
        hs.add("王五");
        hs.add("赵六");
        /*
        * for(int i = 0; i<hs.size;i++) {
        * hs.get(i); 没有索引，报错，不能用for循环遍历
        * }
        */
        for (String s : hs) {
            System.out.print(s + " ");
        }
        System.out.println();
        Iterator<String> it = hs.iterator(); // 获取迭代器对象
        while (it.hasNext()) {
            String str = it.next();
            System.out.print(str + " ");
        }
    }
}

```

运行结果：



```

2266fe9ed5ab435398\redhat.ja
李四 张三 王五 赵六
李四 张三 王五 赵六
PS D:\tang0v0(1)\learn\java>

```

图 3.6运行结果示意图

%-----%

程序2:

```
package com.company.com06;
```

```
/**
```

```
*Set 集合去重原理:
```

```
*1. 调用对象的hashCode方法, 得到哈希值
```

```
*如果哈希值不同, 则直接存储
```

```
*2. 当哈希值相同时, 才会调用equals方法
```

```
*比较结果为false则存储
```

```
*比较结果是true才会当成重复元素, 不存
```

```
*如何把内容相同视为相同元素:
```

```
*解决方案:
```

```
*
```

```
*重写hashCode和equals 方法
```

```
**/
```

```
import java.util.Objects;
```

```
public class Student /* extends Object */ {
```

```
    private String name;
```

```
    private int age;
```

```
    public Student() {
```

```
    }
```

```
    public Student(String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

```
    @Override
```

```
    public boolean equals(Object o) {
```

```
        if (this == o)
```

```
            return true;
```

```
        if (o == null || getClass() != o.getClass())
```

```
            return false;
```

```

        Student student = (Student) o;
        return age == student.age && Objects.equals(name, student.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }

    @Override
    public String toString() {
        return "名字: " + name + ", 年龄=" + age;
    }
}

%-----%
package com.company.com06;

import java.util.HashSet;

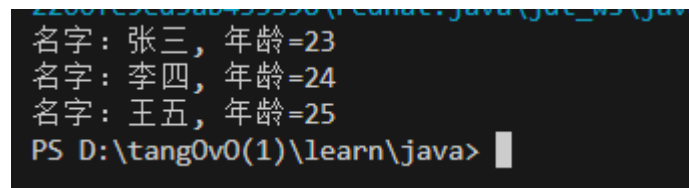
public class HashSetDemo02 {
    public static void main(String[] args) {
        // 注意:如果是自定义类型的对象, 保存到HashSet集合,
        // 必须要重写hashCode()和equals()方法。
        HashSet<Student> hs = new HashSet<>();
        Student s1 = new Student("张三", 23);
        Student s2 = new Student("张三", 23);
        Student s3 = new Student("李四", 24);
        Student s4 = new Student("王五", 25);
        hs.add(s4);
        hs.add(s1);
        hs.add(s2);
        hs.add(s3);
        for (Student student : hs) {
            System.out.println(student);
        }
    }
}

```



```
}
```

运行结果2:



```
22007c9e5ab49990 (r_candc.java) (jre_75) java
名字: 张三, 年龄=23
名字: 李四, 年龄=24
名字: 王五, 年龄=25
PS D:\tang0v0(1)\learn\java>
```

图 3.7运行结果示意图

7、编写程序，学习LinkedHashSet类的用法。

程序:

```
package com.company.com07;
```

```
/**
```

```
* HashSet 集合存储的元素是无序的
```

```
* 想让元素的存取顺序一致，可使用LinkedHashSet类，是HashSet的子类
```

```
* 与LinkedList 一样，它也使用双向链表来维护内部元素的关系。
```

```
*/
```

```
import java.util.Iterator;
```

```
import java.util.LinkedHashSet;
```

```
public class LinkedHashSetDemo {
```

```
    public static void main(String[] args) {
```

```
        LinkedHashSet set = new LinkedHashSet();
```

```
        set.add("张三");
```

```
        // 向该Set集合中添加字符串
```

```
        set.add("李四");
```

```
        set.add("王五");
```

```
        Iterator it = set.iterator(); // 获取 Iterator 对象
```

```
        while (it.hasNext()) {
```

```
            // 通过while循环，判断集合中是否有元素
```

```
            Object obj = it.next();
```

```
            System.out.println(obj);
```

```
        }
```

```
    }
```

```
}
```

运行结果:

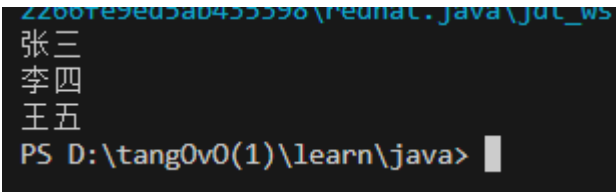


图 3.8运行结果示意图

8、编写程序，演示TreeSet的排序规则，自然排序和自定义排序。

程序1: package com.company.com08;

```
/**
 * TreeSet 集合：底层是二叉树实现。可以对存入的元素进行排序。
 * 排序方式一：自然排序。要求集合中元素的类必须要实现Comparable接口，
 * 重写compareTo编写比较的条件。
 * Java 中大部分的类都实现了Comparable接口，并默认实现了接口中的CompareTo()方法，
 * 如Integer、Double 和 String等。
 * 规则:从小到大
 */
import java.util.TreeSet;

public class TreeSetDemo01 {
    public static void main(String[] args) {
        TreeSet<Integer> ts1 = new TreeSet<>();
        ts1.add(5);
        ts1.add(3);
        ts1.add(2);
        ts1.add(1);
        ts1.add(4);
        System.out.println(ts1);// [1, 2, 3, 4, 5]
        TreeSet<String> ts2 = new TreeSet<>();
        ts2.add("Tom");
        ts2.add("Candy");
        ts2.add("Baby");
        ts2.add("David");
        for (Object o : ts2) {
            System.out.println(o);
        }
    }
}
```

```
}
```

运行结果1:

```
[1, 2, 3, 4, 5]
Baby
Candy
David
Tom
PS D:\tang0v0(1)\learn\java>
```

图 3.9运行结果示意图

程序2: package com.company.com08;

```
/**
```

- * TreeSet集合：底层是二叉树实现。可以对存入的元素进行排序。
 - * 排序方式一：自然排序。要求集合中元素的类必须要实现Comparable接口，
 - * 重写compareTo编写比较的条件。
 - * Java中大部分的类都实现了Comparable接口，并默认实现了接口中的CompareTo()方法，
 - * 如Integer、Double和String 等,规则:从小到大。
 - * 对自定义对象进行排序?
 - * 自定义类：要求存储的元素类必须实现Comparable接口，并重写compareTo()方法。
 - * 可不可以从大到小?
- ```
*/
```

```
public class Student implements Comparable<Student> {
```

```
 private String name;
```

```
 private int age;
```

```
 public Student() {
 }
}
```

```
 public Student(String name, int age) {
 this.name = name;
 this.age = age;
 }
}
```

```
 public String getName() {
 return name;
 }
}
```

```
 public void setName(String name) {
```

```

 this.name = name;
 }

 public int getAge() {
 return age;
 }

 public void setAge(int age) {
 this.age = age;
 }

 @Override
 public String toString() {
 return "name=" + name + ", age=" + age;
 }

 @Override
 public int compareTo(Student o) { // 排序方式一：自然排序
 // 排序规则：按照年龄升序
 int result = this.age - o.getAge(); // o.getAge()-this.age 降序
 /*
 * 编写排序规则返回值特点：
 * 1. 返回的是负数（小）、向左存
 * 2. 返回的是0、不存储
 * 3. 返回的是正数（大）、向右存
 */
 // 次要条件：如果年龄相同、按照姓名排序
 if (result == 0) {
 result = this.name.compareTo(o.getName());
 }
 return result;
 }
}

%-----%

package com.company.com08;

```

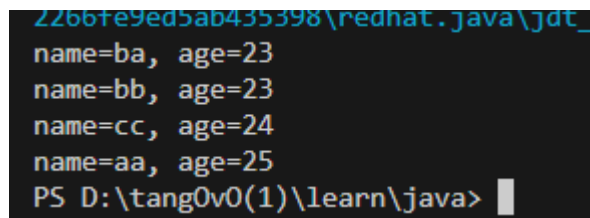
```

import java.util.TreeSet;

/**
 * TreeSet集合：底层是二叉树实现。可以进行元素的排序
 * 自定义类：要求存储的元素类必须实现Comparable接口，并重写compareTo()方法。
 */
public class TreeSetDemo02 {
 public static void main(String[] args) {
 TreeSet<Student> ts = new TreeSet<>();
 Student s1 = new Student("aa", 25);
 Student s2 = new Student("aa", 25);
 Student s3 = new Student("bb", 23);
 Student s4 = new Student("ba", 23);
 Student s5 = new Student("cc", 24);
 ts.add(s1);
 ts.add(s2);
 ts.add(s3);
 ts.add(s4);
 ts.add(s5);
 // System.out.println(ts);
 for (Student student : ts) {
 System.out.println(student);
 }
 }
}

```

运行结果2:



```

2266fe9ed5ab435398\redhat.java\jdk_
name=ba, age=23
name=bb, age=23
name=cc, age=24
name=aa, age=25
PS D:\tang0v0(1)\learn\java>

```

图 3.10运行结果示意图

程序3: package com.company.com08;

```

public class Teacher {
 private String name;
 private int age;
}

```

```

public Teacher() {
}

public Teacher(String name, int age) {
 this.name = name;
 this.age = age;
}

public String getName() {
 return name;
}

public void setName(String name) {
 this.name = name;
}

public int getAge() {
 return age;
}

public void setAge(int age) {
 this.age = age;
}

@Override
public String toString() {
 return "name=' " + name + "', age=" + age;
}
}

%-----%

package com.company.com08;

import java.util.Comparator;
import java.util.TreeSet;

```

```

/**
 * 排序方式二（推荐）：比较器排序。需要在TreeSet集合的构造方法中,传递比较器接口
 * 的实现类对象。编写排序条件！
 *
 * TreeSet 排序：
 * 自然排序：
 * 自定义类 实现 Comparable接口重写compareTo方法并指定排序规则
 * 定制排序：（推荐）
 * 通过TreeSet的有参构造，传递Comparator对象并指定排序规则
 * 注意:定制排序优先级大于自然排序
 */
public class TreeSetDemo03 {
 public static void main(String[] args) {
 TreeSet<Teacher> ts = new TreeSet<Teacher>(new Comparator<Teacher>() {
 @Override
 public int compare(Teacher o1, Teacher o2) {
 // 按照年龄降序排序
 int result = o2.getAge() - o1.getAge();
 return result;
 }
 });
 Teacher t1 = new Teacher("张三", 23);
 Teacher t2 = new Teacher("王五", 25);
 Teacher t3 = new Teacher("李四", 26);
 Teacher t4 = new Teacher("赵四", 24);
 ts.add(t1);
 ts.add(t2);
 ts.add(t3);
 ts.add(t4);
 // System.out.println(ts);
 for (Teacher teacher : ts) {
 System.out.println(teacher);
 }
 }
}

```

运行结果3:

```
name='李四', age=26
name='王五', age=25
name='赵四', age=24
name='张三', age=23
PS D:\tang0v0(1)\learn\java>
```

图 3.11运行结果示意图

9、编写程序，演示Map接口常用方法。

程序: package com.company.com09;

```
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

/**
 * Map 接口方法:
 * void put(Object key, Object value)
 * Object get(Object key)
 * void clear()
 * 添加一对数据
 * 根据键获取值
 * 清空集合元素
 * Object remove(Object key)
 * 根据键删除一对数据，返回被删除的值
 * 54
 * int size()
 * 获取集合中的长度
 * boolean containsKey(Object key) 判断集合中是否包含指定的键
 * boolean containsValue(Object value) 判断集合中是否包含指定的值
 * Collection<V> values()
 * 获取集合中所有的值，保存到单列集合
 * String replace (Object key, Object value) 将集合中 key 所映射的值修改为value
 */
public class MapDemo {
 public static void main(String[] args) {
 // 创建HashMap 集合对象
 // Map<String, String> map = new HashMap<>();
```



```

HashMap<String, String> map = new HashMap<>();
// void put(Object key, Object value) 添加一对数据（键值映射）
// Map 集合中的键是唯一的、值可以重复
map.put("s01", "张三");
map.put("s02", "王五");
map.put("s03", "赵六");
// 使用新值将老值替换。并返回老值
String oldValue = map.put("s01", "李四");
System.out.println(oldValue);
System.out.println(map);
// 有时内容不同，哈希值相同
// System.out.println("重地".hashCode());
// System.out.println("通话".hashCode());
// method01(map);
}

private static void method01(Map<String, String> map) {
 // int size() 获取集合中的长度
 System.out.println(map.size());
 // Object get(Object key) 根据键获取值
 String value = map.get("s02");
 System.out.println(value);
 // boolean containsKey(Object key) 判断集合中是否包含指定的键
 System.out.println(map.containsKey("s02"));
 System.out.println(map.containsKey("s05"));
 // boolean containsValue(Object value) 判断集合中是否包含指定的值
 System.out.println(map.containsValue("王五"));
 System.out.println(map.containsValue("周七"));
 // V remove(Object key) 根据键删除一对数据，返回被删除的值
 String value2 = map.remove("s03");
 System.out.println(value2);
 System.out.println(map);
 // Collection<V> values() 获取集合中所有的值，保存到单列集合
 Collection<String> values = map.values();
 System.out.println(values);
 // String replace (Object key, Object value)

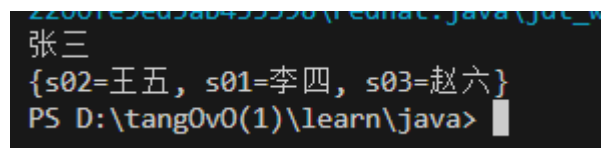
```

```

 // 将集合中key所映射的值修改为value
 map.replace("s02", "田老八");
 System.out.println(map);
 // void clear()清空集合元素
 map.clear();
 System.out.println(map);
 }
}

```

运行结果：



```

张三
{s02=王五, s01=李四, s03=赵六}
PS D:\tang0v0(1)\learn\java>

```

图 3.12运行结果示意图

## 10、编写程序，演示Map集合遍历方式。

程序：package com.company.com10;

```

import java.util.*;

/**
 * Map遍历：
 * map集合是不能直接使用增强for和普通for，迭代器.
 * Map集合遍历方式：
 * 方法一： Set keySet() 获取所有的键、保存到Set集合
 *
 * 1、map 双列集合不能直接遍历
 *
 * 2、调用keySet()获取所有的键，保存到Set集合中，
 * 遍历Set集合可获得所有的键,配合get()方法就可获得键对应的值。
 * 方法二： Set<Map.Entry<K,V>>entrySet() 获取所有键值对对象、保存到Set集合
 *
 * entrySet()方法会将每一对数据，封装成一个一个Entry对象，
 *
 * 然后保存在Set集合中，然后遍历Set集合，得到Entry对象，
 * 通过getKey()和getValue()方法获得键和值。

```

\* 第三种: forEach (BiConsumer) JDK8版本新增的遍历的方法。

\*

\* forEach ();

\* 第四种: values (); 获取map集合中所有的值, 返回到一个Collection集合中。

\*

\*/

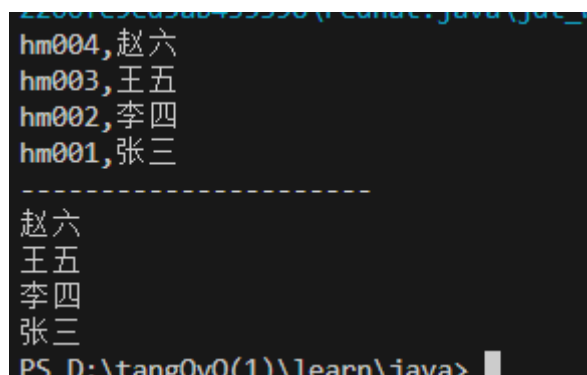
```
public class LinkedHashMapDemo {
 public static void main(String[] args) {
 // 创建HashMap 集合对象
 // Map<String,String> map = new HashMap<>();
 HashMap<String, String> map = new HashMap<>();
 // void put(Object key, Object value) 添加一对数据 (键值映射)
 map.put("hm001", "张三");// Map 集合中的键是唯一的、值可以重复
 map.put("hm002", "李四");
 map.put("hm003", "王五");
 map.put("hm004", "赵六");
 // 第一种方式: keySet() 方法
 // Set keySet() 获取所有的键、保存到Set集合
 Set<String> keys = map.keySet(); // map 双列集合不能直接遍历, 两种方式
 for (String key : keys) {
 String value = map.get(key);
 System.out.println(key + ", " + value);
 }
 System.out.println("-----");
 /*
 * // 第二种方式: entrySet() 方法
 * // Set<Map.Entry<K,V>>entrySet()
 * 获取所有键值对对象、保存到Set集合
 * Set<Map.Entry<String, String>> entries = map.entrySet();
 * for (Map.Entry<String, String> entry : entries) {
 * String key = entry.getKey();
 * String value = entry.getValue();
 * System.out.println(key+", "+value);
 * }
 * System.out.println("-----");
 * Iterator<Map.Entry<String, String>> it = entries.iterator();
```

```

 * while (it.hasNext()) {
 * Map.Entry<String, String> entry = it.next();
 * String key = entry.getKey();
 * String value = entry.getValue();
 * System.out.println(key+", "+value);
 * }
 * System.out.println("-----");
 * //第三种: forEach (BiConsumer) JDK8 版本新增的遍历的方法。
 * map.forEach((String key, String value)->{
 * System.out.println(key+"="+value);
 * });
 * System.out.println("-----");
 */
// 第四种: values (); 获取map集合中所有的值, 返回到一个Collection集合中。
// forEach ();
Collection<String> values = map.values();
values.forEach((String value) -> {
 System.out.println(value);
});
}
}

```

运行结果:



```

hm004, 赵六
hm003, 王五
hm002, 李四
hm001, 张三

赵六
王五
李四
张三
PS D:\tang0v0(1)\learn\java>

```

图 3.13运行结果示意图

11、编写程序, 学习LinkedHashMap集合的用法。

程序: package com.company.com11;

```
import java.util.LinkedHashMap;
```

```
import java.util.Set;
```

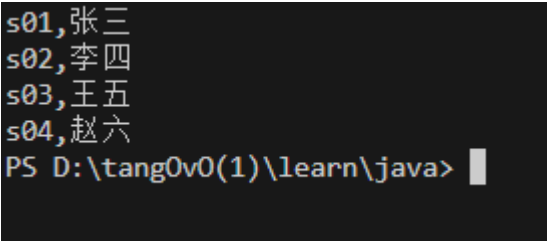
```
public class LinkedHashMapTest {
```

```

public static void main(String[] args) {
 // HashMap<String,String> hm = new HashMap<>(); //存储无序
 // 注意：如果想要保证元素的存取顺序，可以使用LinkedHashMap集合。
 LinkedHashMap<String, String> hm = new java.util.LinkedHashMap<>();
 hm.put("s01", "张三");
 hm.put("s02", "李四");
 hm.put("s03", "王五");
 hm.put("s04", "赵六");
 Set<String> keys = hm.keySet();
 for (String key : keys) {
 String value = hm.get(key);
 System.out.println(key + ", " + value);
 }
}
}

```

运行结果：



```

s01, 张三
s02, 李四
s03, 王五
s04, 赵六
PS D:\tang0v0(1)\learn\java>

```

图 3.14运行结果示意图

12、编写程序，实现TreeMap集合按键值排序，键是自定义类，值是String类。

程序：package com.company.com12;

```

/**
 * 自定义类：要求存储的元素类必须实现Comparable接口，并重写compareTo()方法。
 */
public class Student implements Comparable<Student> {
 private String name;
 private int age;

 public Student() {
 }

 public Student(String name, int age) {
 this.name = name;
 }
}

```

```

 this.age = age;
 }

 public String getName() {
 return name;
 }

 public void setName(String name) {
 this.name = name;
 }

 public int getAge() {
 return age;
 }

 public void setAge(int age) {
 this.age = age;
 }

 @Override
 public String toString() {
 return "name=" + name + ", age=" + age;
 }

 @Override
 public int compareTo(Student o) {
 // 排序规则：按照年龄升序
 int result = o.getAge() - this.age; // this.age-o.getAge();升序
 // 次要条件：如果年龄相同、按照姓名排序
 if (result == 0) {
 result = this.name.compareTo(o.getName());
 }
 return result;
 }
}
%-----%
```

```
package com.company.com12;
```

```
public class Teacher {
```

```
 private String name;
```

```
 private int age;
```

```
 public Teacher() {
```

```
 }
```

```
 public Teacher(String name, int age) {
```

```
 this.name = name;
```

```
 this.age = age;
```

```
 }
```

```
 public String getName() {
```

```
 return name;
```

```
 }
```

```
 public void setName(String name) {
```

```
 this.name = name;
```

```
 }
```

```
 public int getAge() {
```

```
 return age;
```

```
 }
```

```
 public void setAge(int age) {
```

```
 this.age = age;
```

```
 }
```

```
 @Override
```

```
 public String toString() {
```

```
 return "name=" + name + ", age=" + age;
```

```
 }
```

```
}
```

```
%-----%
```

```
package com.company.com12;

import java.util.Comparator;
import java.util.Iterator;
import java.util.Set;
import java.util.TreeMap;

public class TreeMapTest {
 public static void main(String[] args) {
 // 方法1: 自然排序升序, String类已经重写compareTo方法了。
 TreeMap<String, String> tm = new TreeMap<>();
 tm.put("hm002", "张三");
 tm.put("hm003", "李四");
 tm.put("hm001", "王五");
 tm.put("hm004", "赵六");
 System.out.println(tm);
 // 方法1: 重写compareTo方法降序排序。
 TreeMap<Student, String> tm1 = new TreeMap<>();
 Student s1 = new Student("张三", 23);
 Student s2 = new Student("李四", 25);
 Student s3 = new Student("王五", 24);
 tm1.put(s1, "基础班");
 tm1.put(s2, "基础班");
 tm1.put(s3, "就业班");
 System.out.println(tm1);
 // 方法2: 定制排序, 通过Comparator升序排序,
 /*
 * TreeMap<Teacher, String> tm2 = new TreeMap<>();
 * Teacher t1 = new Teacher("张三", 23);
 * Teacher t2 = new Teacher("李四", 25);
 * Teacher t3 = new Teacher("王五", 24);
 * tm2.put(t1, "基础班");
 * tm2.put(t2, "基础班");
 * tm2.put(t3, "就业班");
 * System.out.println(tm2);
 */
 }
}
```

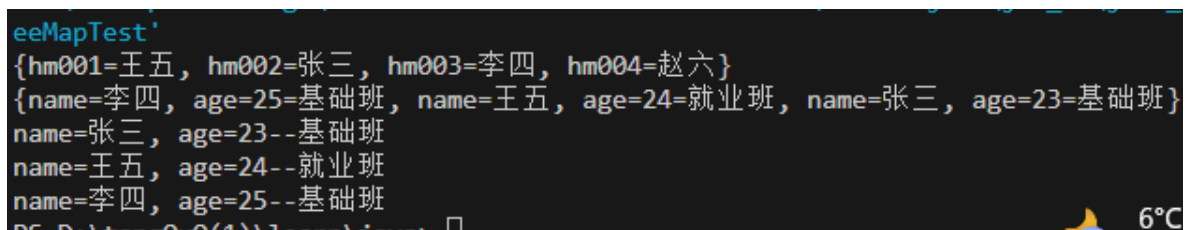


```

 TreeMap<Teacher, String> tm2 = new TreeMap<>(new Comparator<Teacher>()
{
 @Override
 public int compare(Teacher t1, Teacher t2) {
 return t1.getAge() - t2.getAge();
 }
});
Teacher t1 = new Teacher("张三", 23);
Teacher t2 = new Teacher("李四", 25);
Teacher t3 = new Teacher("王五", 24);
tm2.put(t1, "基础班");
tm2.put(t2, "基础班");
tm2.put(t3, "就业班");
// System.out.println(tm2);
Set keySet = tm2.keySet();
Iterator it = keySet.iterator();
while (it.hasNext()) {
 Object key = it.next();
 Object value = tm2.get(key); // 获取每个键所对应的值
 System.out.println(key + "--" + value);
}
}
}

```

运行结果:



```

eeMapTest'
{hm001=王五, hm002=张三, hm003=李四, hm004=赵六}
{name=李四, age=25=基础班, name=王五, age=24=就业班, name=张三, age=23=基础班}
name=张三, age=23--基础班
name=王五, age=24--就业班
name=李四, age=25--基础班

```

图 3.15运行结果示意图

13、编写程序，演示Collections类中常用的方法。

程序: package com.company.com13;

```

import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;

```

```

/**
 * Collections 集合工具类:
 * static <T> boolean addAll(Collection<? super T> c, T...elements) 将所有指定元
 * 素添加到指定集合c中
 * static void reverse(List list) 反转指定List集合中元素的顺序
 * static void shuffle(List list) 对 List集合中的元素进行随机排序
 * static void sort(List list) 根据元素的自然顺序对List集合中的元素进行排序
 * static void swap(List list,int i,int j) 将指定List集合中角标i处元素和j处元
 * 素进行交换
 * 使用二分法搜索指定对象在List集合中的索引, 查找的List集合中的元素必须是有序的
 * static int binarySearch(List list,Object key)
 * static Object max(Collection col) 根据元素的自然顺序, 返回给定集合中最大的元素
 * static Object min(Collection col) 根据元素的自然顺序, 返回给定集合中最小的元素
 * 用一个新值newVal替换List集合中所有的旧值oldVal
 * static boolean replaceAll(List list,Object oldVal,Object newVal)
 */
public class CollectionsDemo {
 public static void main(String[] args) {
 ArrayList<String> list = new ArrayList<>();
 // static <T> boolean addAll(Collection<? super T> c, T...elements)
 // 将所有指定元素添加到指定集合c中
 Collections.addAll(list, "a", "b", "c", "d");
 System.out.println(list);
 // static void reverse(List list)反转指定 List 集合中元素的顺序
 Collections.reverse(list);
 System.out.println(list);
 // static void shuffle(List list)对 List 集合中的元素进行随机排序
 Collections.shuffle(list);
 System.out.println(list);
 // sort(List list)根据元素的自然顺序对List集合中的元素进行排序
 Collections.sort(list);
 System.out.println(list);
 // swap(List list,int i,int j) 将指定 List集合中角标i处元素和j处元素进
 行交换
 // Collections.swap(list,2,3);
 // System.out.println(list);
 }
}

```

是有序的

// 使用二分法搜索指定对象在List集合中的索引，查找的List集合中的元素必须是有序的

// static int binarySearch(List list, Object key)

int index = Collections.binarySearch(list, "c");

System.out.println(index);

最大的元素

// static Object max(Collection col) 根据元素的自然顺序，返回给定集合中最大的元素

String max = Collections.max(list);

System.out.println(max);

最小的元素

// static Object min(Collection col) 根据元素的自然顺序，返回给定集合中最小的元素

String min = Collections.min(list);

System.out.println(min);

// 用一个新值newVal替换List集合中所有的旧值oldVal

// static boolean replaceAll(List list, Object oldVal, Object newVal)

Collections.replaceAll(list, "b", "f");

System.out.println(list);

}

}

运行结果：

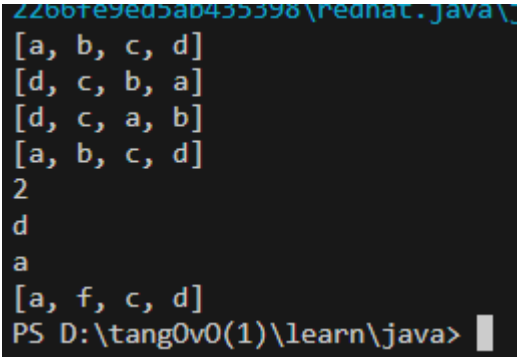


图 3.16运行结果示意图

14、编写程序，演示Arrays工具类中常用的方法。

程序：package com.company.com14;

```
import java.util.Arrays;

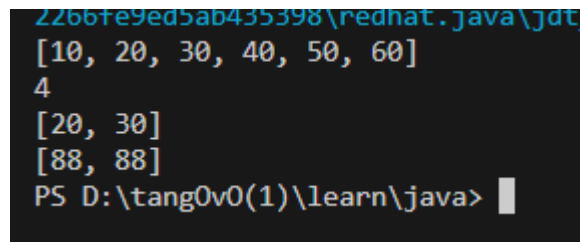
/**
 * Arrays 数组工具类：
 * void sort(); 数组排序；
 * String toString(); 打印数组
 * int binarySearch(Object[] a, Object key) 用二分查找法，查找元素，返回索引位置
```

```

 * int[] copyOfRange(int[] original,int from,int to) 复制数组元素到一个新数组中
 * void fill(Object[] a, Object val) 传入的元素替换数组中所有的元素
 */
public class ArraysDemo {
 public static void main(String[] args) {
 int[] arr = { 10, 50, 30, 20, 40, 60 };
 // void sort(); 数组排序
 Arrays.sort(arr);
 // String toString(); 打印数组
 System.out.println(Arrays.toString(arr));
 // int binarySearch(Object[] a, Object key)
 // 使用二分查找法, 查找元素, 返回索引位置
 int index = Arrays.binarySearch(arr, 50);
 System.out.println(index);
 // int[] copyOfRange(int[] original, int from, int to)
 // 复制数组元素到一个新数组中
 int[] arr2 = Arrays.copyOfRange(arr, 1, 3); // 不包含结束索引
 System.out.println(Arrays.toString(arr2));
 // void fill(Object[] a, Object val) 传入的元素替换数组中所有的元素
 Arrays.fill(arr2, 88);
 System.out.println(Arrays.toString(arr2));
 }
}

```

运行结果:



```

2266fe9ed5ab435398\redhat.java\jdk_
[10, 20, 30, 40, 50, 60]
4
[20, 30]
[88, 88]
PS D:\tangOv0(1)\learn\java>

```

图 3.17运行结果示意图

15、编写程序, 演示Lambda表达式的使用。

程序1: package com.company.com15;

```

/**
 * Lambda 表达式:简化代码书写
 * 1. 使用前提

```

- \* 必须是接口。接口中只能有一个抽象方法, 对使用接口中方法的一种优化。
- \* 2. 使用格式
- \* (数据类型 参数名1, 数据类型 参数名2----) -> {方法体 }
- \* () : 代表的是要执行接口中对应的那个重写方法。如果方法中有参数, 那么就需要传递参数。
- \* -> : 固定格式。作用是将小括号中的参数传递给后面的大括号方法体中。
- \* {} : 代表的是实现接口中那个抽象方法后的方法体。
- \* 匿名内部类: 没有名字的特殊局部内部类
- \* 格式:
- \* new 类名 或 接口名 () {重写方法}
- \* 使用场景: 作为方法的参数传递
- \* 传统实现方式:
- \* 1. 编写实现类
- \* 2. 重写抽象方法
- \* 3. 创建实现类对象
- \* 4. 将实现类对象作为方法的参数传递
- \*/

```
public class LambdaDemo01 {
 public static void main(String[] args) {
 useInter(new Inter() {
 @Override
 public void show(String str) {
 System.out.println(str);
 }
 });
 System.out.println("-----");
 useInter(str -> System.out.println(str));
 }

 // 使用Inter接口的方法
 public static void useInter(Inter i) {
 i.show("hello");
 }
}
```

```
interface Inter {
```

```
public abstract void show(String str);
}
```

运行结果1:

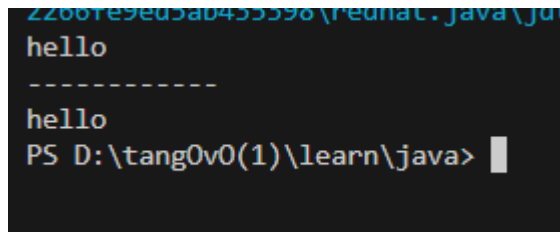


图 3.18运行结果示意图

程序2: package com.company.com15;

```
import java.util.Comparator;
import java.util.TreeSet;

public class LambdaDemo02 {
 public static void main(String[] args) {
 /*
 * TreeSet<Integer> ts = new TreeSet<>(new Comparator<Integer>() {
 * //ALT+回车用 Lambda 替换
 *
 * @Override
 * public int compare(Integer o1, Integer o2) {
 * return o2-o1;
 * }
 * });
 */
 TreeSet<Integer> ts = new TreeSet<>((o1, o2) -> o2 - o1);
 // 多个参数，数据类型可以不写，如(Integer o1, Integer o2)->{return o2-o1;}
 // 可以简化写法
 ts.add(5);
 ts.add(2);
 ts.add(3);
 ts.add(1);
 ts.add(4);
 System.out.println(ts);
 }
}
```

}

运行结果2:

```
2266fe9ed5ab435398\redhat.java\jdt
[5, 4, 3, 2, 1]
PS D:\tang0v0(1)\learn\java>
```

图 3.19运行结果示意图

三、上机体会

通过本次实验了解到了集合的概念和大体明白了集合用于做什么，初步掌握了Collection接口、List接口、Set接口、Map接口在常用方法的使用，与此同时掌握List接口中的ArrayList、LinkedList、Iterator和foreach循环的使用，以及初步掌握了Set接口中的HashSet、LinkedHashSet和TreeSet的使用，理解了TreeMap集合的键值排序，还有对于Arrays，Lambda的表达式使用。

# 滇池学院理工学院

## 实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 图形用户界面开发

实验时间 2025 年 11 月 14 .21.2 8 日 第 34 节

年级 2024 专业班级 计科1班

姓名 唐明强 学 号 20242122002

评分 [单击此处成绩](#)

王庆平

教师签名



# 上机四 图形用户界面开发

## 一、上机目的

- 1、了解GUI的概念
- 2、掌握AWT包中主要类的层次结构
- 3、掌握常用的容器种类
- 4、掌握布局管理器的种类
- 5、掌握事件处理的机制及适配器类
- 6、了解Swing包中主要类的层次结构。
- 7、掌握Swing常用的容器。
- 8、掌握Swing中组件的使用方法。

## 二、上机任务

### 1、窗体和面板。

定义一个公共类FramePanel继承Frame，在main()方法中创建FramePanel对象frame，然后设置frame为可见及其大小，并取消布局管理器，最后创建Panel类对象pan，设置pan的大小和背景颜色，并将其添加到frame中。

#### 程序1：（在主函数中直接创建框架对象的方式）

```
package com.company01;

/*
 * 在主函数中直接创建框架对象的方式
 * 优点：简洁直接，无需额外定义类，代码量少；
 * 缺点：主函数臃肿，所有逻辑（配置、组件、事件）堆叠。
 * 适用场景：1）小型 Demo、快速原型验证；
 *2）仅需显示简单窗口，无复杂功能；
 *3）新手学习框架API的基础使用。
 */
import java.awt.*;

public class FramePanelTest {
 public static void main(String[] args) {
 // 创建Frame类对象frame，设置标题为“TextEditor”
 Frame frame = new Frame("TextEditor");
 frame.setVisible(true); // 调用setVisible()方法，设置窗体可见
 frame.setSize(240, 150); // 调用setSize()方法，设置窗体大小
 frame.setLayout(null); // 调用setLayout()方法，取消布局管理器
```

```

 Panel pan = new Panel();
 // 创建Panel类对象pan
 pan.setSize(100, 100);
 // 调用setSize()方法，设置面板大小
 // 调用setBackground()方法，设置面板背景颜色为红色
 pan.setBackground(Color.red);
 frame.add(pan);
 // 调用add()方法把面板添加到窗体中
 }
}

```

## 程序2：（使用创建框架子类的方式）

```
package com.company01;
```

```
/*
```

```
* 优点：结构清晰，初始化、组件、事件分离封装；
```

```
* 缺点：需额外定义子类，少量增加代码量。
```

```
* 框架子类方式：1）中大型应用程序；
```

```
*2）需要自定义窗口功能（如绘图、事件处理）；
```

```
*3）多个地方复用相同窗口；
```

```
*5）追求代码可维护性和扩展性。
```

```
*/
```

```
import java.awt.*;
```

```
// 定义FramePanel类，继承Frame类
```

```
public class FramePanel extends Frame {
```

```
 // 定义有参构造方法，传入标题
```

```
 public FramePanel(String title) {
```

```
 super(title);
```

```
 // 调用父类的构造方法
```

```
 setVisible(true); // 调用setVisible()方法，设置窗体可见
```

```
 setSize(240, 150); // 调用setSize()方法，设置窗体大小
```

```
 setLayout(null); // 调用setLayout()方法，取消布局管理器
```

```
 // setBackground(Color.blue); // 窗体背景颜色为蓝色
```

```
 }
```

```
}
```

```
%-----%
```

```

package com.company01;

import java.awt.*;

public class FramePanelTest {

 public static void main(String[] args) {

 // 创建FramePanel类对象frame，设置标题为“TextEditor”
 FramePanel frame = new FramePanel("TextEditor");

 Panel pan = new Panel(); // 创建Panel类对象pan
 pan.setSize(100, 100);

 // 调用setSize()方法，设置面板大小
 // 调用setBackground()方法，设置面板背景颜色为红色
 pan.setBackground(Color.red);

 frame.add(pan); // 调用add()方法把面板添加到窗体中
 }
}

```

运行结果：

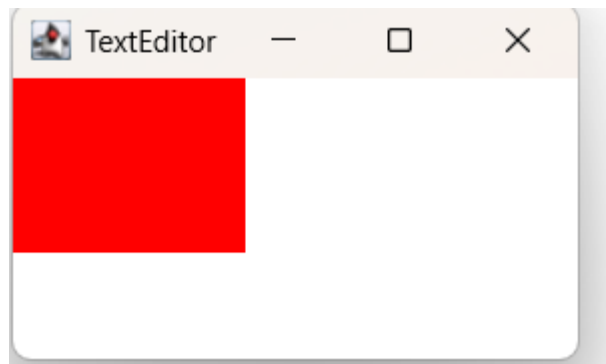


图 4.1 运行结果示意图

2、编写程序， 创建一个用户登录界面，实现输入用户名和密码（设置回显字符为“\*”）后，单击“确定”按钮输出用户名和密码。 注意设置参数：-Dfile.encoding=gbk。

程序：package com.company02;

```

import java.awt.*;
import java.awt.event.*;

public class UserLogin extends Frame implements ActionListener {

 // 创建用户名标签lName，并设置标签文本为“用户名”
 Label lName = new Label("用户名");

 // 创建用户名文本框tName
 TextField tName = new TextField();

 // 创建密码标签lPass，并设置标签文本为“密码”
 Label lPass = new Label("密码");
}

```

```
// 创建密码文本框tPass
TextField tPass = new TextField();

// 创建确定按钮btnConfirm，并设置标签文本为“确定”
Button btnConfirm = new Button("Enter");

public UserLogin(String title) {
 // 调用父类构造方法，设置窗体标题
 super(title);
 // 设置窗体大小
 setSize(300, 200);
 // 取消窗体的布局管理器
 setLayout(null);
 // 设置用户名标签与文本框的位置与大小
 lName.setBounds(60, 50, 70, 20);
 tName.setBounds(135, 50, 100, 20);
 // 设置密码标签与文本框的位置与大小
 lPass.setBounds(60, 90, 70, 20);
 tPass.setBounds(135, 90, 100, 20);
 tPass.setEchoChar('*');
 // 设置确定按钮的位置和大小
 // 设置密码文本框的回显字符
 btnConfirm.setBounds(110, 140, 70, 20);
 // 为确定按钮注册事件侦听器
 btnConfirm.addActionListener(this);
 // 将组件添加到窗体中
 add(lName);
 add(tName);
 add(lPass);
 add(tPass);
 add(btnConfirm);
 // 设置窗体的位置与可见性
 setLocation(200, 100);
 setVisible(true);
}

// 事件处理方法
```

```
public void actionPerformed(ActionEvent e) {
 // 获取事件对象
 Object ob = e.getSource();
 // 如果事件对象为btnConfirm，则输出用户名和密码
 if (ob == btnConfirm) {
 System.out.println("用户名: " + tName.getText());
 System.out.println("密码: " + tPass.getText());
 }
}

}

}

%-----%

package com.company02;

public class UserLoginTest {
 public static void main(String[] args) {
 // 创建UserLogin类对象
 new UserLogin("登录窗口");
 }
}

}
```

运行结果：

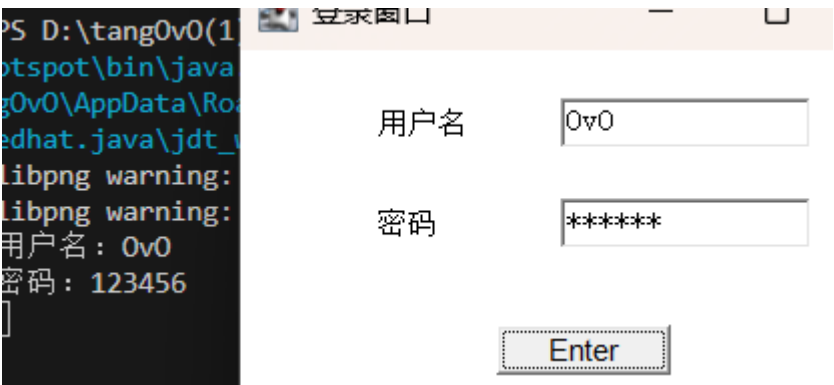


图 4.2 运行结果示意图

3、编写程序，创建一个个人信息调查窗口，显示姓名、性别、年龄、籍贯、爱好和喜欢的网站等信息，效果如下图所示。



**程序:** package com.company03;

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class Questionnaire extends Frame implements ActionListener {
 // 创建姓名文本框，文本框宽度为10
 TextField name = new TextField(10);
 CheckboxGroup sex = new CheckboxGroup();
 // 创建性别单选按钮组
 Checkbox man = new Checkbox("男", true, sex); // 创建男单选按钮
 Checkbox woman = new Checkbox("女", false, sex); // 创建女单选按钮
 // 创建年龄文本框，初始值为20，文本框宽度为4
 TextField age = new TextField("20", 4);
 Choice nativeplace = new Choice(); // 创建籍贯下拉式菜单
 Checkbox like1 = new Checkbox("读书");
 // 创建读书复选框
 Checkbox like2 = new Checkbox("上网");
 // 创建上网复选框
 Checkbox like3 = new Checkbox("体育活动");// 创建体育活动复选框
 List website = new List(4);
 // 创建喜欢的网站列表，显示4行
 Button btnOK = new Button("确认");
 // 创建确认按钮
 Button btnCancel = new Button("取消"); // 创建取消按钮
 Button btnQuit = new Button("退出");
 // 创建退出按钮
 // 创建标签
 Label label = new Label("个人信息调查表");
 Label lName = new Label("姓名");
 Label lSex = new Label("性别");
 Label lAge = new Label("年龄");
 Label lNativeplace = new Label("籍贯");
 Label lLike = new Label("爱好");
 Label lWebsite = new Label("喜欢的网站");
```

```
public Questionnaire(String title) {
 super(title);
 // 调用父类构造方法
 setSize(400, 400);
 setLayout(null);
 // 设置组件的位置和大小
 // 设置窗体的大小
 // 取消窗体的布局管理器
 label.setBounds(150, 50, 100, 20);
 lName.setBounds(50, 100, 40, 20);
 name.setBounds(90, 100, 100, 20);
 lSex.setBounds(230, 100, 40, 20);
 man.setBounds(270, 100, 60, 20);
 woman.setBounds(330, 100, 60, 20);
 lAge.setBounds(50, 150, 40, 20);
 age.setBounds(90, 150, 50, 20);
 // 个人信息调查表标签
 // 姓名标签
 // 姓名文本框
 // 性别标签
 // 男单选钮
 // 女单选钮
 // 年龄标签
 // 年龄文本框
 lNativeplace.setBounds(230, 150, 40, 20); // 籍贯标签
 nativeplace.setBounds(270, 150, 60, 20); // 籍贯选项框
 // 设置下拉菜单选项, 添加城市名
 nativeplace.add("北京");
 nativeplace.add("上海");
 nativeplace.add("天津");
 nativeplace.add("重庆");
 nativeplace.add("广东");
 nativeplace.add("河南");
 // 设置组件的位置和大小
 lLike.setBounds(50, 200, 40, 20);
 like1.setBounds(90, 200, 60, 20);
```

```
like2.setBounds(150, 200, 60, 20);
like3.setBounds(210, 200, 100, 20);
// 爱好标签
// 读书标签
// 上网标签
// 体育活动标签
lWebsite.setBounds(50, 250, 80, 20); // 喜欢的网站标签
website.setBounds(130, 250, 100, 60); // 喜欢的网站列表
// 设置列表选项，添加网站名
website.add("新浪");
website.add("搜狐");
website.add("网易");
website.add("淘宝");
website.add("赶集网");
website.add("新华网");
// 设置组件的位置和大小
btnOK.setBounds(110, 330, 50, 20);
// 确认按钮
btnCancel.setBounds(180, 330, 50, 20); // 取消按钮
btnQuit.setBounds(250, 330, 50, 20);
// 退出按钮
// 将各组件添加到窗体中
add(label);
add(lName);
add(name);
add(lSex);
add(man);
add(woman);
add(lAge);
add(age);
add(lNativeplace);
add(nativeplace);
add(lLike);
add(like1);
add(like2);
add(like3);
```



```

 add(lWebsite);
 add(website);
 add(btnOK);
 add(btnCancel);
 add(btnQuit);
 setLocationRelativeTo(null);
 setVisible(true);
 // 使窗体在屏幕上居中放置
 // 设置窗体可见
 // 为三个按钮注册事件侦听器
 btnOK.addActionListener(this);
 btnCancel.addActionListener(this);
 btnQuit.addActionListener(this);
 }

 // 重写事件处理方法
 public void actionPerformed(ActionEvent e) {
 Object ob = e.getSource();
 // 获取事件对象
 // 单击退出按钮
 // 退出系统
 // 单击确认按钮
 if (ob == btnQuit) {
 System.exit(0);
 } else if (ob == btnOK) {
 // 输出信息
 System.out.println("姓名: " + name.getText());
 System.out.println("性别: " +
sex.getSelectedCheckbox().getLabel());
 System.out.println("年龄: " + age.getText());
 System.out.println("籍贯: " + nativeplace.getSelectedItem());
 // 如果复选框被选中, 则返回其标签, 否则将字符串设置为空
 String s1 = like1.getState() ? like1.getLabel() + " " : "";
 String s2 = like2.getState() ? like2.getLabel() + " " : "";
 String s3 = like3.getState() ? like3.getLabel() + " " : "";
 System.out.println("爱好: " + s1 + s2 + s3);
 }
 }

```

```
 System.out.println("喜欢的网站: " + website.getSelectedIndex());
 } else if (ob == btnCancel) {
 name.setText("");
 // 单击取消按钮
 // 清空姓名文本框

 sex.setSelectedCheckbox(man); // 选中“男”单选钮
 age.setText("20");
 // 设置年龄文本框为20
 // 取消爱好复选框
 like1.setState(false);
 like2.setState(false);
 like3.setState(false);
 // 取消所选喜欢的网站
 website.deselect(website.getSelectedIndex());
 }
}

public static void main(String[] args) {
 new Questionnaire("个人信息");
}
}
```

运行结果：

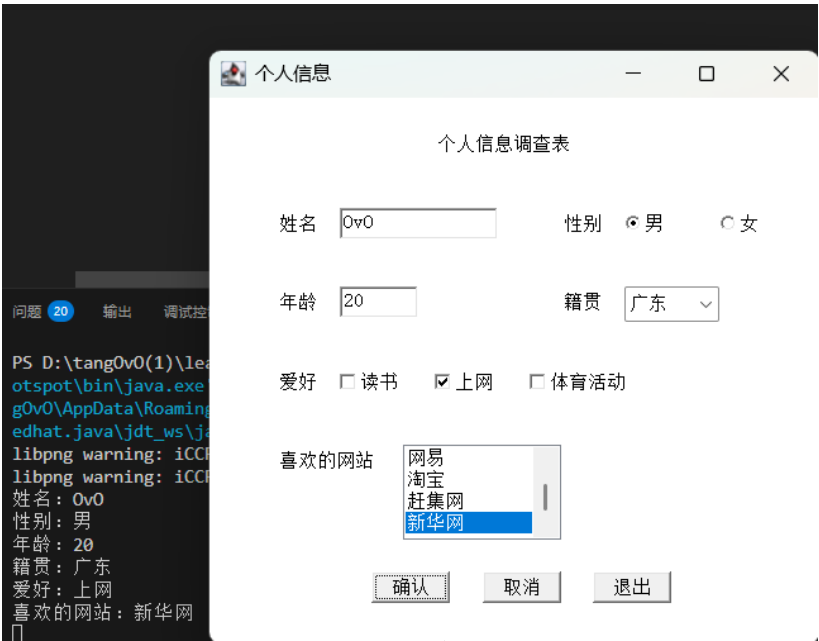


图 4.3 运行结果示意图

4、编写程序，窗口中放置一个文本区和一个按钮。两个组件都处理鼠标事件，当鼠标在文本区单击时，文本区以红色字体显示鼠标单击的位置坐标；鼠标单击按钮时，文本区以蓝色字

体显示鼠标单击的位置坐标，并且窗体能够响应关闭操作。

程序：package com.company04;

```
import java.awt.*;

import java.awt.event.*;

public class EventAdapter extends Frame {

 private TextArea txtApp = new TextArea("这是文本区");

 private Button btnApp = new Button("这是按钮");

 // 创建鼠标事件侦听器

 MouseEventHandler handler = new MouseEventHandler();

 public EventAdapter(String title) {

 super(title);

 // 为窗体注册窗口事件侦听器

 addWindowListener(new WindowAdapter() {

 // 重写windowClosing()方法，关闭当前窗口

 public void windowClosing(WindowEvent e) {

 System.exit(0);

 }

 });

 // 设置窗体的大小，取消其布局管理器，并使窗体在屏幕上居中放置

 setSize(300, 200);

 setLayout(null);

 setLocationRelativeTo(null);

 // 设置文本区的字体、位置和大小

 txtApp.setFont(new Font("宋体", Font.PLAIN, 16));

 txtApp.setBounds(10, 40, 280, 100);

 // 为文本区注册鼠标事件侦听器

 txtApp.addMouseListener(handler);

 // 设置按钮的位置和大小

 btnApp.setBounds(120, 150, 60, 30);

 // 为按钮注册鼠标事件侦听器

 btnApp.addMouseListener(handler);

 // 将文本区和按钮添加到窗体中

 add(txtApp);
```

```

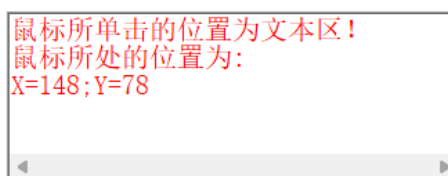
 add(btnApp);
 // 显示窗体
 setVisible(true);
 }

 public static void main(String args[]) {
 new EventAdapter("Adapter Demo");
 }

 // 定义继承MouseAdapter适配器的事件侦听器类
 class MouseEventHandler extends MouseAdapter {
 public void mousePressed(MouseEvent e) {
 // 检测事件源是文本区还是按钮
 if (e.getSource() == txtApp) {
 String s = "鼠标所单击的位置为文本区! \n";
 s += "鼠标所处的位置为:\nX=" + e.getX() + ";Y=" + e.getY();
 // 用红色字体显示鼠标单击的位置坐标
 txtApp.setForeground(Color.RED);
 txtApp.setText(s);
 }
 if (e.getSource() == btnApp) {
 String s = "鼠标所单击的位置为按钮! \n";
 s += "鼠标所处的位置为:\nX=" + e.getX() + ";Y=" + e.getY();
 // 用蓝色字体显示鼠标单击的位置坐标
 txtApp.setForeground(Color.BLUE);
 txtApp.setText(s);
 }
 }
 }
}

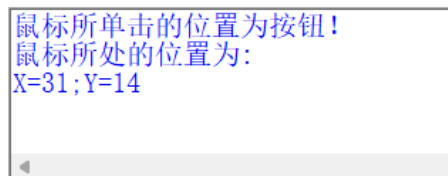
```

运行结果:



鼠标所单击的位置为文本区!  
鼠标所处的位置为:  
X=148;Y=78

这是按钮



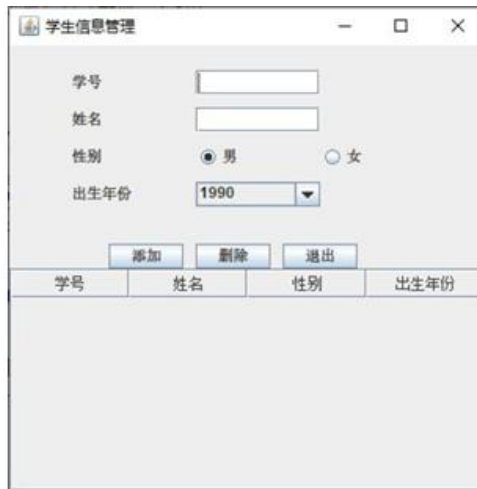
鼠标所单击的位置为按钮!  
鼠标所处的位置为:  
X=31;Y=14

这是按钮

图

#### 4.4 运行结果示意图

5、创建学生信息管理系统，实现添加、删除和退出功能。单击“添加”按钮将信息添加到表格中并显示，信息包括学号、姓名、性别和出生年份；单击“删除”按钮可删除表格中的选择项；单击“退出”按钮关闭系统。



程序：package com.company05;

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class StudentManager extends JFrame implements ActionListener {
 JTextField tNo = new JTextField(10);
 // 创建学号文本框tNo

 JTextField tName = new JTextField(10); // 创建姓名文本框tName
 // 创建性别单选按钮组groupSex，包含两个单选按钮

 ButtonGroup groupSex = new ButtonGroup();
 JRadioButton man = new JRadioButton("男", true);
 JRadioButton woman = new JRadioButton("女");
 // 创建出生年份下拉列表框cbAge

 JComboBox<String> cbAge = new JComboBox<String>();
 JButton btnOK = new JButton("添加");
 // 创建添加按钮btnOK

 JButton btnDelete = new JButton("删除"); // 创建删除按钮btnDelete
 JButton btnQuit = new JButton("退出"); // 创建退出按钮btnQuit

 JLabel lNo = new JLabel("学号");
 // 创建学号标签lNo
```

```
JLabel lName = new JLabel("姓名");
JLabel lSex = new JLabel("性别");
JLabel lAge = new JLabel("出生年份");
JTable table;
// 创建姓名标签lName
// 创建性别标签lSex
// 创建出生年份标签lAge
// 声明表格table
DefaultTableModel model; // 创建 DefaultTableModel 类对象 model

public StudentManager(String title) {
 super(title);
 // 调用父类构造方法
 setSize(400, 400);
 // 设置组件位置和大小
 // 设置窗体的大小
 lNo.setBounds(50, 20, 100, 20);
 tNo.setBounds(150, 20, 100, 20);
 lName.setBounds(50, 50, 100, 20);
 tName.setBounds(150, 50, 100, 20);
 lSex.setBounds(50, 80, 100, 20);
 man.setBounds(150, 80, 60, 20);
 woman.setBounds(250, 80, 60, 20);
 lAge.setBounds(50, 110, 100, 20);
 cbAge.setBounds(150, 110, 100, 20);
 // 向groupSex 组添加单选钮man和woman
 groupSex.add(man);
 groupSex.add(woman);
 // 设置下拉列表框选项, 添加年份
 cbAge.addItem("1990");
 cbAge.addItem("1989");
 cbAge.addItem("1988");
 cbAge.addItem("1987");
 cbAge.addItem("1986");
 cbAge.addItem("1985");
 cbAge.addItem("1984");
```

```
// 设置3个按钮位置和大小
btnOK.setBounds(80, 160, 60, 20);
btnDelete.setBounds(150, 160, 60, 20);
btnQuit.setBounds(220, 160, 60, 20);
Container c = getContentPane(); // 创建 Container 类对象 c
JPanel panel = new JPanel();
// 创建面板对象panel
panel.setLayout(null);
// 将各组件添加到面板panel中
panel.add(tNo);
panel.add(tName);
panel.add(man);
panel.add(woman);
panel.add(cbAge);
panel.add(lNo);
panel.add(lName);
panel.add(lSex);
panel.add(lAge);
panel.add(btnOK);
panel.add(btnDelete);
panel.add(btnQuit);
// 取消panel的布局管理器
String[] colName = { "学号", "姓名", "性别", "出生年份" };
// 创建model
model = new DefaultTableModel(colName, 0);
// 创建table, 并使用model保存table数据
table = new JTable(model);
// 创建滚动面板scrollPane, 将table添加到scrollPane中
JScrollPane scrollPane = new JScrollPane(table);
// 设置窗体的布局管理器为GridLayout, 分为2行1列
c.setLayout(new GridLayout(2, 1));
c.add(panel);
c.add(scrollPane);
setLocationRelativeTo(null);
setVisible(true);
// 将panel添加到窗体中
```

```

 // 将scrollPane 添加到窗体中
 // 使窗体在屏幕上居中放置
 // 设置窗体可见
 // 为三个按钮注册事件侦听器
 btnOK.addActionListener(this);
 btnDelete.addActionListener(this);
 btnQuit.addActionListener(this);
 }

 // 重写ActionEvent事件处理方法
 public void actionPerformed(ActionEvent e) {
 Object ob = e.getSource(); // 获取事件对象
 if (ob == btnQuit) {
 // 单击退出按钮
 System.exit(0);
 } else if (ob == btnOK) {
 // 退出系统
 // 单击确认按钮
 // 根据选择获取性别
 String sex;
 if (man.isSelected())
 sex = "男";
 else
 sex = "女";
 String[] stuInfo = { tNo.getText(), tName.getText(),
 sex, cbAge.getSelectedItem().toString() };
 model.addRow(stuInfo);
 // 将信息添加到model中
 tNo.setText("");
 tName.setText("");
 man.setSelected(true);
 // 清空学号文本框
 // 清空姓名文本框
 // 选中man单选钮
 } else if (ob == btnDelete) { // 单击删除按钮
 if (table.getSelectedRow() < 0)

```



```

 // 弹出一个错误提示对话框
 JOptionPane.showMessageDialog(null, "请在表格中选择要删除的
选项",

 "警告", JOptionPane.WARNING_MESSAGE);

 else

 // 删除选择项
 model.removeRow(table.getSelectedRow());
 }

}

public static void main(String[] args) {
 new StudentManager("学生信息管理");
}
}
```

运行结果：



图 4.5 运行结果示意图

6、创建一个简易文本编辑器，包括一个主菜单和一个文本区。其中，主菜单包括“新建文件”“打开文件”“关闭文件”“保存文件”和“退出”选项。

程序：package com.company06;

```

import java.awt.*;
import java.awt.event.*;
import java.io.*;
import javax.swing.*;

public class TextEditor extends JFrame implements ActionListener {
```

```

JMenuBar mainMenubar; // 声明菜单栏

JMenu file;

// 声明主菜单项

JMenuItem nw;

JMenuItem op;

JMenuItem cl;

JMenuItem sf;

JMenuItem ex;

JTextArea tx;

// 声明新建文件子菜单项
// 声明打开文件子菜单项
// 声明关闭文件子菜单项
// 声明保存文件子菜单项
// 声明退出子菜单项
// 声明文本区
public TextEditor(String title) {
 super(title);
 // 调用父类构造方法
 setSize(400, 400); // 设置窗体大小
 setLocationRelativeTo(null); // 使窗体在屏幕上居中放置
 menuinit();
 // 调用构建与处理菜单的方法
 tx = new JTextArea();
 add(tx);
 setVisible(true);
}

// 定义菜单构建与处理方法
void menuinit() {
 // 创建文本区对象
 // 将文本区对象放入窗体
 // 使窗体可见
 mainMenubar = new JMenuBar(); // 创建主菜单栏
 file = new JMenu("文件");
 // 创建主菜单项

```

```

// 创建各子菜单项
nw = new JMenuItem("新建文件");
op = new JMenuItem("打开文件");
cl = new JMenuItem("关闭文件");
sf = new JMenuItem("保存文件");
ex = new JMenuItem("退出");
// 将各子菜单项加入到主菜单项中
file.add(nw);
file.add(op);
file.add(cl);
file.add(sf);
file.add(ex);
mainMenubar.add(file);
// 将主菜单项加入到主菜单栏
setJMenuBar(mainMenubar); // 为窗体设置主菜单
// 为各子菜单项注册事件侦听器
nw.addActionListener(this);
op.addActionListener(this);
cl.addActionListener(this);
sf.addActionListener(this);
ex.addActionListener(this);
}

// 重写ActionEvent事件处理方法
public void actionPerformed(ActionEvent e) {
 Object ob = e.getSource();
 // 获取事件对象
 JFileChooser f = new JFileChooser(); // 创建文件选择器对象
 // 选择"新建文件"或"关闭文件"子菜单项
 if ((ob == nw) || (ob == cl)) {
 tx.setText("");
 // 清空文本区
 } else if (ob == op) { // 选择"打开文件"子菜单项
 // 弹出具有自定义按钮的文件选择器对话框
 f.showOpenDialog(this);
 try {

```

```

 // 定义StringBuffer 对象s
 StringBuffer s = new StringBuffer();
 // 创建FileReader对象in, 参数为在文件选择器中选中的文件
 FileReader in = new FileReader(f.getSelectedFile());
 // 读取文件内容, 将其追加到s中
 while (true) {
 int b = in.read();
 if (b == -1)
 break;
 s.append((char) b);
 }
 tx.setText(s.toString()); // 将文件内容显示在文本区
 in.close();
 // 关闭文件
 } catch (Exception e1) {
 }
} else if (ob == sf) {
 // 选择"保存文件"子菜单项
 f.showSaveDialog(this); // 显示文件选择对话框
 try {
 // 创建FileWriter对象, 其参数为选择的文件
 FileWriter out = new FileWriter(f.getSelectedFile());
 out.write(tx.getText()); // 将文本区内容写入文件
 out.close();
 // 关闭文件
 } catch (Exception e2) {
 }
} else if (ob == ex) // 选择"退出"子菜单项
 System.exit(0);
// 退出系统
}

public static void main(String[] args) {
 new TextEditor("简易文本编辑器");
}
}

```

运行结果:

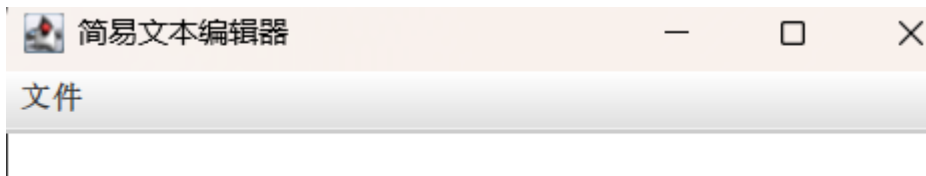


图 4.6 运行结果示意图

7. 利用BoxLayout 布局，设计登录界面。

程序: package com.company06;

```
import java.awt.*;
import javax.swing.*;

class BoxLayoutDemo extends JFrame {
 private JLabel jLabel1, jLabel2, jLabel3;
 private JButton jConnect;
 private JTextField jUID;
 private JPasswordField jPwd, jConfirmPwd;

 BoxLayoutDemo() {
 super("用户注册界面");
 jLabel1 = new JLabel("用户名: ");
 jLabel2 = new JLabel("密码: ");
 jLabel3 = new JLabel("确认密码: ");
 jConnect = new JButton("注册");
 jUID = new JTextField(15);
 jPwd = new JPasswordField(15);
 jConfirmPwd = new JPasswordField(15);
 Box userName = Box.createHorizontalBox();
 // 创建Box容器的水平格式
 Box password = Box.createHorizontalBox();
 Box confirmPassword = Box.createHorizontalBox();
 Box submitButton = Box.createHorizontalBox();
 userName.add(jLabel1);
 // 在Box容器中加入组件
 userName.add(jUID);
 password.add(jLabel2);
```

```

password.add(jPwd);
confirmPassword.add(jLabel3);
confirmPassword.add(jConfirmPwd);
submitButton.add(jConnect);
this.setLayout(new BorderLayout(this.getContentPane(), BorderLayout.Y_AXIS));
// 设置BoxLayout 布局
this.add(userName);
this.add(password);
this.add(confirmPassword);
this.add(submitButton);
userName.setVisible(true);
password.setVisible(true);
confirmPassword.setVisible(true);
submitButton.setVisible(true);
this.setSize(240, 150);
this.setVisible(true);
this.setDefaultCloseOperation(EXIT_ON_CLOSE);
}

public static void main(String[] args) {
 new BoxLayoutDemo();
}
}

```

运行结果：

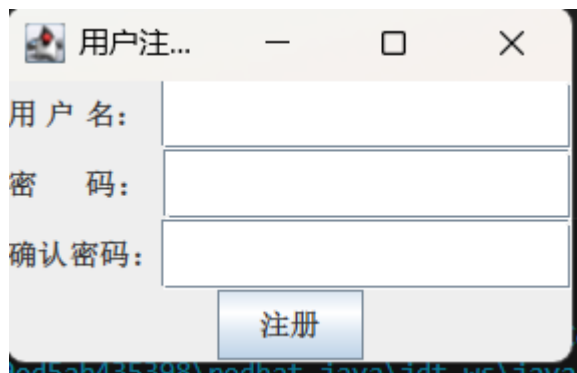


图 4.7 运行结果示意图

### 三、上机体会

通过本次实验了解GUI的概念，掌握AWT包中主要类的层次结构，掌握常用的容器种类，掌握布局管理器的种类，掌握事件处理的机制及适配器类，了解Swing包中主要类的层次结构。掌握Swing常用的容器，掌握Swing中组件的使用方法。

# 滇池学院理工学院

## 实验报告

课程名称 面向对象程序设计二

课程性质 实训课

实验名称 I/O 流和文件操作

实验时间 2025 年 11 月 21.2 5 日 第 34 节

年级 2024 专业班级 计科 1 班

姓名 唐明强 学 号 20242122002

评分 [单击此处成绩](#)

王庆平

教师签名

## 上机五 I/O 流和文件操作

### 一、上机目的

- 1、了解流的分类；
- 2、掌握 Java 程序运行时的 3 个系统流；
- 3、掌握使用 File 类的构造方法创建文件对象，掌握 File 类中操作文件的常用方法；
- 4、了解字节输入流、输出流的层次结构和常用方法；
- 5、掌握文件字节输入/输出流的构造方法，缓存字节输入/输出流的构造方法，据字节输入/输出流的构造方法；
- 6、了解字符输入/输出流的层次结构和常用方法；
- 7、掌握文件字符输入/输出流类的构造方法，掌握缓存字符输入/输出流类的构造方法；
- 8、了解 RandomAccessFile 类与输入/输出流类的区别；
- 9、掌握 RandomAccessFile 类的常用方法；
- 10、通过上机实操，将所学知识与实际应用相结合。

### 二、上机任务

#### 1、文件操作。

**要求：**定义文件操作类 FileOperation，在 main() 方法中定义一个 File 对象 dir（目录）和一个 File 对象数组 file（文件）；然后判断 dir 目录和 file 文件是否存在，如果不存在则创建；接着重命名 file 中的第一个文件，并输出 dir 下的所有文件名；最后删除 file 中的文件，并输出 dir 下的所有文件名。

**程序：**

```
package com.test1;
import java.io.*;

public class FileOperation {
 public static void main(String[] args) throws IOException {
 // 创建File对象dir，表示目录
 File dir = new File("D:/source");
 if (!dir.exists()) {
 dir.mkdir();
 }
 File[] file = new File[5];
 // 如果目录不存在
 // 创建目录
 for (int i = 0; i < file.length; i++) {
 // 定义文件数组file
 // 循环
 // 定义文件名，并以test1~test5命名文件
 String fileName = String.format("test%d.txt", i+1);
 // 在dir下创建File对象，并添加到file中，表示文件
 file[i] = new File(dir, fileName);
 if (!file[i].exists()) {
 // 如果文件不存在
 try {
 file[i].createNewFile();
 } catch (IOException e) {
 // 创建一个新文件
 // 捕获IOException异常
 System.out.println(e.getMessage());
 }
 }
 }
 // 重命名文件数组中的第一个文件
```

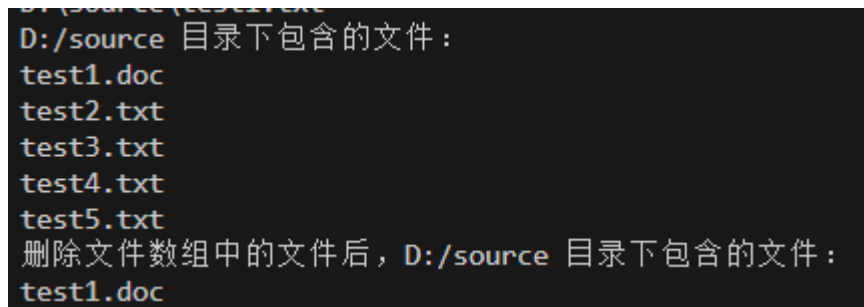


```

 file[0].renameTo(new File(dir, "test1.doc"));
 System.out.println(file[0]);
 String[] list = dir.list();
// 获取dir下的所有文件名
 System.out.println("D:/source 目录下包含的文件：");
 for (int i = 0; i < list.length; i++)
 System.out.println(list[i]);
// 删除文件数组中的所有文件
 for (int i = 0; i < file.length; i++) {
 if (file[i].exists())
 file[i].delete();
 }
 list = dir.list();
// 获取删除文件后dir下的所有文件名
 System.out.println("删除文件数组中的文件后，" +
 "D:/source 目录下包含的文件：");
// 输出删除文件后dir下的所有文件名
 for (int i = 0; i < list.length; i++)
 System.out.println(list[i]);
 }
}

```

运行结果：



```

D:/source 目录下包含的文件：
test1.doc
test2.txt
test3.txt
test4.txt
test5.txt
删除文件数组中的文件后，D:/source 目录下包含的文件：
test1.doc

```

图 5.1 运行结果示意图

2、使用文件字节输入/输出流读写文件。 要求：定义FileSteamTest 类，在 main()方法中创建 FileOutputStream 类对象，并使用 write()方法向 FileSteamTest.txt 文件中写入字符串，写入前须使用getBytes()方法将字符串转化成字节数组；然后创建 FileInputStream类对象，并使用 read()方法读取文件中的数据，并将读取的字节数组转化成字符串后输出。

程序：

```

package com.test2;
import java.io.*;

public class FileSteamTest {
 public static void main(String[] args) {
 // 定义要写入的字符串
 String content = "这是一个使用文件字节流进行读写操作的测试文件。\\nHello,
FileInputStream and FileOutputStream!";

 // 定义文件名
 String fileName = "FileSteamTest.txt";

 // 使用FileOutputStream写入文件
 try (FileOutputStream fos = new FileOutputStream(fileName)) {
 // 将字符串转换为字节数组

```

```

byte[] bytes = content.getBytes();

// 使用write()方法将字节数组写入文件
fos.write(bytes);

System.out.println("数据已成功写入文件：" + fileName);
System.out.println("写入的内容：" + content);

} catch (IOException e) {
 System.out.println("写入文件时发生错误：" + e.getMessage());
 return;
}

// 使用FileInputStream读取文件
try (FileInputStream fis = new FileInputStream(fileName)) {
 // 创建字节数组来存储读取的数据
 byte[] buffer = new byte[fis.available()];

 // 使用read()方法读取文件数据到字节数组
 int bytesRead = fis.read(buffer);

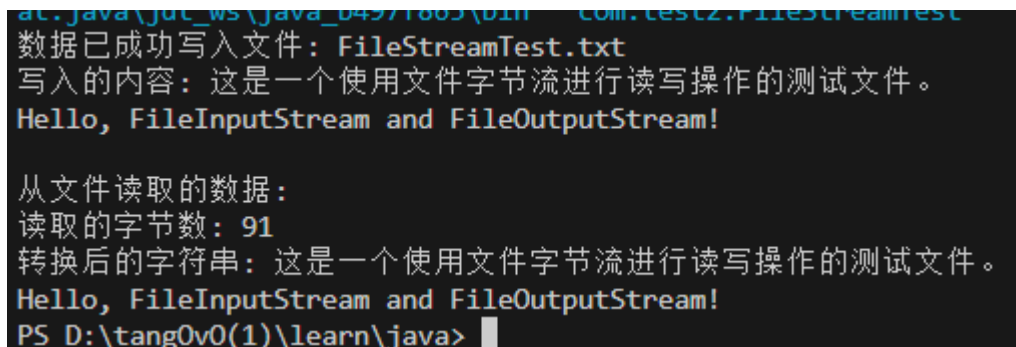
 // 将字节数组转换为字符串
 String readContent = new String(buffer, 0, bytesRead);

 System.out.println("\n从文件读取的数据:");
 System.out.println("读取的字节数：" + bytesRead);
 System.out.println("转换后的字符串：" + readContent);

} catch (IOException e) {
 System.out.println("读取文件时发生错误：" + e.getMessage());
}
}
}

```

运行结果:



```

at.java(jdt_ws(java_04571883\bin com.test2.FileStreamTest
数据已成功写入文件: FileStreamTest.txt
写入的内容: 这是一个使用文件字节流进行读写操作的测试文件。
Hello, FileInputStream and FileOutputStream!

从文件读取的数据:
读取的字节数: 91
转换后的字符串: 这是一个使用文件字节流进行读写操作的测试文件。
Hello, FileInputStream and FileOutputStream!
PS D:\tang0v0(1)\learn\java>

```

图 5.2 运行结果示意图

3、使用缓存字节输入/输出流读写文件。 要求: 使用BufferedOutputStream 类创建对象bos (参数为FileOutputStream 类对象) 向文件 写入数据, 然后使用BufferedInputStream 类创建对象bis (参数为FileInputStream 类对象) 从 文件读取数据。

程序:

```

package com.test3;
import java.io.*;

public class BufferedStreamTest {
 public static void main(String[] args) {

```

```

 // 定义要写入的字符串
 String content = "这是一个使用缓存字节流进行读写操作的测试文件。\\nHello,
 BufferedOutputStream and BufferedInputStream!";

 // 定义文件名
 String fileName = "BufferedStreamTest.txt";

 // 使用BufferedOutputStream写入文件
 try (FileOutputStream fos = new FileOutputStream(fileName);
 BufferedOutputStream bos = new BufferedOutputStream(fos)) {

 // 将字符串转换为字节数组
 byte[] bytes = content.getBytes();

 // 使用write()方法将字节数组写入缓存流
 bos.write(bytes);

 // 刷新缓存，确保数据写入文件
 bos.flush();

 System.out.println("数据已成功写入文件：" + fileName);
 System.out.println("写入的内容：" + content);

 } catch (IOException e) {
 System.out.println("写入文件时发生错误：" + e.getMessage());
 return;
 }

 // 使用BufferedInputStream读取文件
 try (FileInputStream fis = new FileInputStream(fileName);
 BufferedInputStream bis = new BufferedInputStream(fis)) {

 // 创建字节数组来存储读取的数据
 byte[] buffer = new byte[bis.available()];

 // 使用read()方法从缓存流读取数据到字节数组
 int bytesRead = bis.read(buffer);

 // 将字节数组转换为字符串
 String readContent = new String(buffer, 0, bytesRead);

 System.out.println("\\n从文件读取的数据：");
 System.out.println("读取的字节数：" + bytesRead);
 System.out.println("转换后的字符串：" + readContent);

 } catch (IOException e) {
 System.out.println("读取文件时发生错误：" + e.getMessage());
 }
 }
}

```

运行结果：

```
at.java\jdk_ws\java_b497f865\bin com.test3.BufferedStreamTest
数据已成功写入文件：BufferedStreamTest.txt
写入的内容：这是一个使用缓存字节流进行读写操作的测试文件。
Hello, BufferedOutputStream and BufferedInputStream!

从文件读取的数据：
读取的字节数：99
转换后的字符串：这是一个使用缓存字节流进行读写操作的测试文件。
Hello, BufferedOutputStream and BufferedInputStream!
PS D:\tang0v0(1)\learn\java> █
```

图 5.3 运行结果示意图

三、上机体会

通过本次实验了解流的分类；掌握Java程序运行时的3个系统流；掌握使用File类的构造方法创建文件对象，掌握File类中操作文件的常用方法；了解字节输入流、输出流的层次结构和常用方法；掌握文件字节输入/输出流的构造方法，缓存字节输入/输出流的构造方法，据字节输入/输出流的构造方法；了解字符输入/输出流的层次结构和常用方法；掌握文件字符输入/输出流类的构造方法，掌握缓存字符输入/输出流类的构造方法；了解RandomAccessFile 类与输入/输出流类的区别；掌握RandomAccessFile 类的常用方法;通过上机实操，将所学知识与实际应用相结合。

# 实验报告

教师签名

# 上机六 多线程

## 一、上机目的

- 1、了解进程与线程,能够说出进程与线程的区别;
- 2、掌握使用Thread类、Runnable接口和Callable接口实现多线程;
- 3、熟悉后台线程的使用,能够理解后台线程用于做什么;
- 4、了解线程的生命周期及状态转换;
- 5、能够说出线程的生命周期的6种基本状态以及这6种状态的转换方式;
- 6、掌握操作线程的相关方法;
- 7、学会正确使用线程的优先级、休眠、合并、让步和中断操作;
- 8、掌握多线程的同步,能够正确地使多线程同步。

## 二、上机任务 1、继承Thread类创建多线程。

程序:

```
package com.company.text1;

public class MyThread extends Thread {
 // 线程要执行代码
 @Override
 public void run() {
 for (int i = 0; i < 10; i++) {
 System.out.println("子线程:" + i);
 }
 }
}

%-----%

package com.company.text1;

public class Test01 {
 public static void main(String[] args) {
 MyThread thread = new MyThread();
 thread.start();
 for (int i = 0; i < 10; i++) {
 System.out.println("main:" + i);
 }
 }
}
```

运行结果:

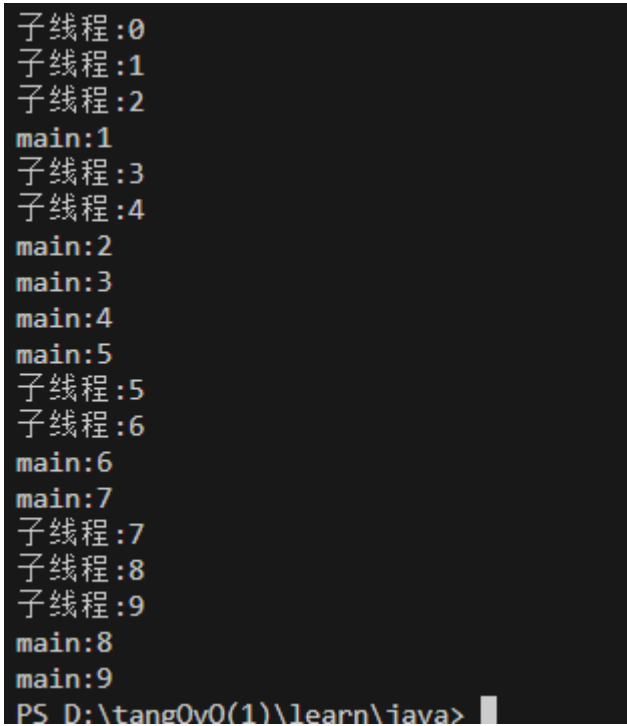


图 6.1 运行结果示意图

2、实现Runnable接口创建多线程。

程序:

```
package com.company.text2;

public class Test02 {
 public static void main(String[] args) {
 MyRunnable myRunnable = new MyRunnable();
 Thread thread = new Thread(myRunnable);
 thread.start();
 for (int i = 0; i < 10; i++) {
 System.out.println("main:" + i);
 }
 }
}

%-----%

package com.company.text2;

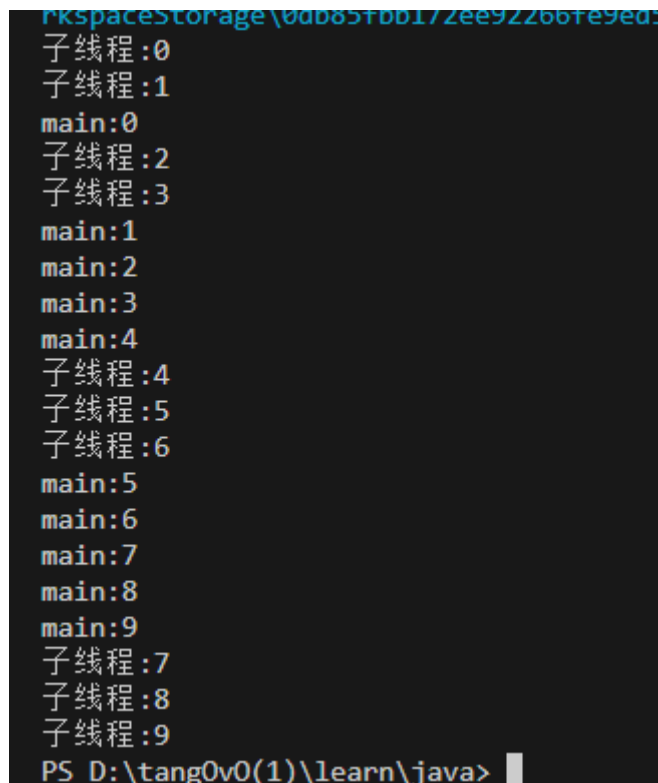
public class MyRunnable implements Runnable {
 @Override
 public void run() {
 for (int i = 0; i < 10; i++) {
```

```

 System.out.println("子线程:" + i);
 }
}
}
}

```

运行结果:



```

rkspacestorage\0db85fbb172ee92266fe9ed3
子线程:0
子线程:1
main:0
子线程:2
子线程:3
main:1
main:2
main:3
main:4
子线程:4
子线程:5
子线程:6
main:5
main:6
main:7
main:8
main:9
子线程:7
子线程:8
子线程:9
PS D:\tang0v0(1)\learn\java>

```

图 6.2 运行结果示意图

### 3、基于Callable接口和FutureTask类实现多线程。

程序: package com.company.text3;

```
import java.util.concurrent.*;
```

```
public class Test03 {
```

```
 public static void main(String[] args) throws ExecutionException,
```

```
 InterruptedException {
```

```
 MyCallable mc = new MyCallable();
```

```
 FutureTask<String> ft = new FutureTask<>(mc);
```

```
 Thread thread = new Thread(ft); // FutureTask 是 Runnable 实现类
```

```
 thread.start();
```

```
 // 用于获取执行结果, 该方法会发生阻塞, 一直等到任务执行完毕才返回执行结
```

果

```
 String s = ft.get();
```

```
 System.out.println(s);
```



```
 }
}
%-----%
package com.company.text3;

import java.util.concurrent.Callable;

public class MyCallable implements Callable<String> {
 @Override
 public String call() throws Exception {
 for (int i = 0; i < 5; i++) {
 System.out.println("子线程表白第: " + (i + 1) + "次");
 }
 return "答应了!!!";
 }
}
```

运行结果:

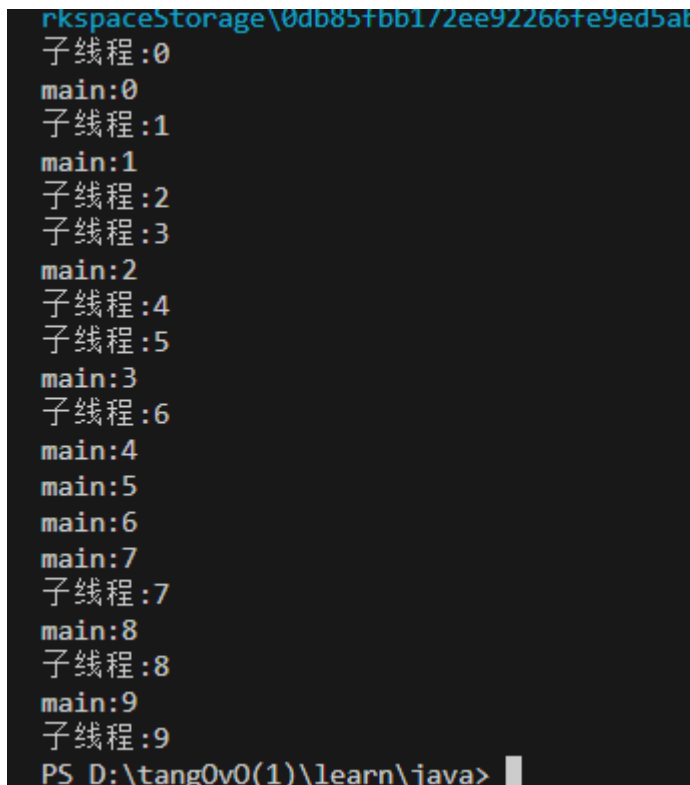


图 6.3 运行结果示意图

### 三、上机体会

Java 作为一门广泛应用的编程语言，其课程通常具有以下特点： 1、入门相对友好：语法结构清晰严谨，接近自然语言，对于初学者来说较易理解和上手，适 合编程入门学习。

2、面向对象思想贯穿：课程会重点讲解封装、继承、多态三大面向对象特性，培养学习者以对象为中心的编程思维，这也是 Java 编程的核心思想。

3、平台无关性突出：“一次编写，到处运行”（Write Once, Run Anywhere）是 Java 的重要特性，课程中会介绍 JVM（Java 虚拟机）的作用，解释其跨平台原理。

4、实践性强：课程往往包含大量的编程练习和项目实践，从简单的控制台程序（如计算器、学生信息管理）到复杂的应用开发，强调通过实践掌握知识。

5、内容体系庞大：涵盖基础语法（变量、数据类型、控制流等）、集合框架、I/O 流、多线程、网络编程、GUI 编程（如 Swing、JavaFX）、数据库操作（JDBC）等多个方面，知识点丰富且系统性强。

6、与实际应用结合紧密：很多课程会结合 Java 在企业级开发（如 Spring 框架）、移动应用开发（Android）等领域的应用场景进行讲解，让学习者了解知识的实际用途。

7、强调代码规范：课程中会注重培养学习者编写规范、易读、可维护代码的习惯，这对于团队协作和后续开发至关重要。

8、异常处理机制完善：会详细讲解异常的概念、分类以及处理方式（try-catch-finally、throws），培养学习者应对程序错误的能力。这些特点使得 Java 课程既能帮助初学者打下扎实的编程基础，也能为深入学习高级开发技术奠定良好的基础。

学习 Java 课程需要结合理论理解、大量实践和思维培养，以下是一些高效的学习方法：

1、打牢基础，循序渐进 Java 基础语法（变量、数据类型、运算符、控制流等）是后续学习的基石，务必扎实掌握。可以按“语法规则→简单案例→小练习”的节奏推进，比如先理解 if-else 逻辑，再用它实现一个成绩评级程序，避免跳过基础直接学习框架。

2、吃透面向对象思想 面向对象是 Java 的核心，要重点理解封装、继承、多态的设计理念：封装：思考“为什么要将属性私有化，通过方法访问？”继承：分析“父类和子类的关系，如何避免过度继承？”多态：结合接口和抽象类，理解“如何通过统一接口实现不同功能”。可以通过设计简单类（如“动物-猫-狗”的继承关系）加深理解。

3、边学边练，注重实践 每学一个知识点，立即写代码验证：比如学了 ArrayList，就用它实现一个“待办事项列表”，练习增删改查。解决实际问题：尝试用 Java 写一个计算器、学生信息管理系统等小项目，将零散知识点串联起来。模仿与创新：先看懂别人的代码（如 JDK 源码中的简单类），再尝试修改或重写，培养代码逻辑。

4、善用调试和排错 遇到报错不要怕，学会使用 IDE（如 IntelliJ IDEA）的调试工具（断点、单步执行）跟踪代码执行流程，观察变量变化。同时，认真分析错误信息（如 NullPointerException、ClassCastException），理解错误原因比直接复制解决方案更重要。

5、阅读源码和文档 JDK 源码是最好的学习资料，比如 ArrayList、HashMap 的实现，能帮助理解数据结构和设计模式。学会查阅官方文档（Java SE Documentation），培养自主解决问题的能力，这是程序员的必备技能。

6、参与项目开发 当基础扎实后，通过完整项目（如简易博客系统、图书管理系统）提升综合能力，涉及 I/O 流、数据库连接（JDBC）、多线程等知识。可以先跟着教程做，再尝试独立开发，最后优化代码（如添加异常处理、改进架构）。

7、培养编程思维 学习 Java 不仅是记语法，更要培养“如何把现实问题转化为代码”的思维：遇到问题先拆解：比如“实现用户登录”可拆分为“输入验证→数据库查询→结果反馈”。学习设计模式：如单例模式、工厂模式，理解“为什么这样设计”，而不是死记代码。

8、定期复习和总结 用思维导图梳理知识点（如集合框架的体系、异常处理的流程），定期回顾易错点（如值传递与引用传递的区别、静态方法与实例方法的使用场景）。同时，记录自己的代码和思路，形成个人笔记，方便后续查阅。

9、利用资源，拓展学习 优质课程：MOOC（如中国大学 MOOC 的 Java 课程）、国外教程（如 Oracle 的官方教程）。社区交流：遇到问题可在 Stack Overflow、掘金等平台提问或搜索，了解他人的解决方案。关注行业动态：了解 Java 在企业开发（Spring 生态）、Android 开发等领域的应用，明确学习方向。总之，Java 学习没有捷径，关键是“理解原理 + 大量实践 + 持续思考”，从“能写出代码”到“能写出好代码”，逐步提升编程能力。